

✓ Riconoscimento di animali dalle tracce audio (.wav)

Flavio Simonelli 0365333

1. Traccia T4 (Riconoscimento di animali da tracce audio)

Si consideri un dataset contenente registrazioni audio. Si richiede di classificare le tracce riconoscendo la specie animale che l'ha prodotta. Il dataset fornito (cats dogs dolphins.zip) contiene suoni etichettati di 3 specie animali (i.e., cani, gatti e delfini). Si richiede di sviluppare e confrontare 2 diversi modelli:

- un modello che prenda in input direttamente la forma d'onda associata a ciascuna traccia (i.e., sequenze)
- un modello che prenda in input lo spettrogramma di ciascuna traccia, ovvero la rappresentazione grafica dell'intensità di un suono in funzione del tempo e della frequenza.

✓ 2. Analisi Teorica del Problema

Il problema della classificazione dei versi degli animali può essere considerato un'estensione del riconoscimento del parlato umano, adattata all'analisi di segnali audio non verbali.

Le principali caratteristiche acustiche che distinguono i versi degli animali includono:

- **Frequenza dominante e spettro di frequenza:** identificano univocamente la firma acustica di ciascun animale;
- **Cadenza e periodicità del segnale audio:** contribuisce alla discriminazione tramite i diversi pattern vocali.

In questo capitolo si fa riferimento a diversi studi che analizzano le principali tipologie di vocalizzazioni per ciascuna delle tre specie considerate. Oltre a caratteristiche generali (come il significato semantico del verso, l'energia RMS e i coefficienti MFCC) nei seguenti riferimenti vengono analizzate le sue caratteristiche fondamentali prima elencate. In questo capitolo ne viene riportato un breve riassunto.

2.1. Vocalizzazioni dei Delfini

I delfini emettono principalmente tre tipi di suoni:

- **Fischi (Whistles):** spesso utilizzati per la comunicazione.
- **Clic (Clicks):** impiegati per l'ecolocalizzazione.

- **BPS (Burst Pulse Sounds):** sequenze rapide di click che sembrano avere anche una funzione comunicativa.

Caratteristiche principali

Caratteristica	Fischi (Whistles)	Clic (Clicks)	BPS (Burst Pulse Sounds)
Tipo di suono	Tonale e modulato	Impulsivo e ad alta frequenza	Sequenza rapida di impulsi
Frequenza fondamentale	2 kHz - 24 kHz	0,2 kHz - 150 kHz	0,2 kHz - 16 kHz
Durata	0,05 s - 2 s	impulsivi	media 0,1 s
Spettrogramma	Linee curve o ondulate	Impulsi verticali	Sequenze di impulsi ravvicinati

Fonte: [Università di Bari](#)

2.2. Vocalizzazioni dei Gatti

I gatti emettono principalmente tre tipi di suoni:

- **Miagolii:** spesso utilizzati per la comunicazione.
- **Fusa:** suoni continui e ritmici emessi quando il gatto è rilassato o in cerca di conforto.
- **Sibili:** suoni acuti e prolungati emessi in situazioni di difesa o minaccia.

Caratteristiche principali

Caratteristica	Miagolii	Fusa	Sibili
Tipo di suono	Tonale	Ritmico	Acuto e prolungato
Frequenza fondamentale	300 Hz - 6 kHz	20 Hz - 150 Hz	3 kHz - 8 kHz
Durata	Variabile, da breve a prolungata	Lunga e costante	Breve o prolungata
Spettrogramma	Linee curve o irregolari	Banda costante	Linee acute e nette

Fonte: [Wikipedia](#)

2.3. Vocalizzazioni dei cani

I cani emettono principalmente tre tipi di suoni:

- **Abbai:** utilizzati per la comunicazione.
- **Guaiti:** suoni acuti ed espressivi.
- **Ringhi:** suoni gutturali e continui, spesso utilizzati per avvertire o mostrare aggressività.

Caratteristiche principali

Caratteristica	Abbai	Guaiti	Ringhi
Tipo di suono	Breve e ripetitivo	Acuto e lamentoso	Gutturale e continuo
Frequenza fondamentale	400 Hz - 7 kHz	600 Hz - 8 kHz	100 Hz - 1 kHz
Durata del suono	Breve, generalmente < 1 s	Variabile, da breve a prolungata	Lunga e costante
Spettrogramma	Impulsi ripetuti e ben separati	Linee acute e irregolari	Banda continua e densa

✓ 3. Analisi operativa del dataset

Dopo aver inquadrato il problema attraverso un'analisi teorica e individuato le informazioni chiave su cui il nostro modello può basarsi per la classificazione, possiamo procedere con un'analisi operativa del dataset fornito.

In questo capitolo esaminiamo i campioni audio presenti nel dataset, focalizzandoci sulle principali caratteristiche distintive, quali:

- **Durata dei campioni:** parametro utile per comprendere la variabilità temporale dei versi;
- **Frequenza di campionamento:** elemento determinante per la qualità e la risoluzione dell'analisi spettrale.

Questa fase di esplorazione preliminare è essenziale per comprendere la struttura del dataset e individuare eventuali pre-elaborazioni necessarie prima della fase di modellazione.

✓ 3.1. Librerie utilizzate

In questo capitolo vengono utilizzate diverse librerie per la gestione, l'analisi e la visualizzazione dei dati. Di seguito, un elenco delle principali librerie impiegate e il loro utilizzo:

- **os:** consente di navigare tra le cartelle del dataset e gestire i file in modo programmatico.
- **librosa:** utilizzato per il caricamento dei campioni audio senza perdita di informazioni sul sample rate e per la visualizzazione di forme d'onda e spettrogrammi direttamente nel notebook.
- **defaultdict:** una variante dei dizionari Python che semplifica la lettura del codice, creando automaticamente una nuova chiave se questa non esiste.
- **pandas:** utilizzato per la creazione e gestione di DataFrame, facilitando l'analisi strutturata delle informazioni sui campioni audio.
- **matplotlib** e **seaborn:** impiegati per la generazione di grafici e la visualizzazione delle distribuzioni dei dati.
- **numpy:** utilizzato per la manipolazione e l'elaborazione dei campioni audio.
- **soundfile:** necessario per la codifica e la gestione di file audio in formato WAV.
- **IPython.display:** per ascoltare un audio direttamente dal notebook

Queste librerie consentono un'analisi approfondita e una gestione efficace del dataset, facilitando la fase di pre-elaborazione e visualizzazione dei dati.

```
import os
import librosa
from collections import defaultdict
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

```
import soundfile as sf
import IPython.display as ipd
```

✓ 3.2. Caricamento del Dataset

La prima operazione da effettuare nel caso in cui il dataset è all'interno di google drive è eseguire il seguente codice.

```
from google.colab import drive
drive.mount('/content/drive')
```

 Mounted at /content/drive

Identifichiamo il percorso della directory di training.

```
# path della directory di training
ORIGINAL_TRAIN_PATH = "/content/drive/MyDrive/Dataset/cats_dogs_dolphins/train"

# Controllo sull'esistenza della directory
if not os.path.exists(ORIGINAL_TRAIN_PATH):
    print(f"Attenzione: la directory {ORIGINAL_TRAIN_PATH} non esiste!")
```

Per gestire il dataset audio, definiamo una funzione che ne effettua il caricamento in un dizionario a partire dalla directory principale.

A differenza di `tf.keras.utils.audio_dataset_from_directory`, questa implementazione permette di accedere a informazioni fondamentali sui campioni, come il **sample rate**, essenziale per un'analisi approfondita dei segnali audio.

L'organizzazione dei dati in un **dizionario** facilita il successivo pre-processing e l'estrazione delle feature, garantendo maggiore flessibilità nella gestione del dataset.

Anche in questa funzione si assume che il dataset sia organizzato nella seguente struttura:

```
main_directory/
...class_a/
.....a_audio_1.wav
.....a_audio_2.wav
...class_b/
.....b_audio_1.wav
.....b_audio_2.wav
```

```
# Carica i file audio da una directory strutturata in classi e li memorizza in un
def load_from_directory(dir, dictionary):
    for label in os.listdir(dir): # itera su ogni sottocartella
```

```

label_path = os.path.join(dir, label)
if os.path.isdir(label_path):
    for file in os.listdir(label_path): # Itera su ogni file nella sottocarte
        if file.lower().endswith('.wav'): # Filtra per file audio (estensione)
            file_path = os.path.join(label_path, file)
            audio, sr = librosa.load(file_path, sr=None) # Carica il file man
            dictionary[label].append((audio, sr)) # Aggiungi una tupla nel di

```

Utilizziamo `load_from_directory` per caricare il dataset di training nel dizionario `train_dict`.

Viene utilizzato un `defaultdict` solo per facilitare la lettura del codice (non richiede la creazione delle chiavi prima di inserire gli elementi nel dizionario)

```

train_dict = defaultdict(list) # dizionario dataset originale di training

load_from_directory(ORIGINAL_TRAIN_PATH, train_dict)

```

Per analizzare i campioni caricati stampiamo delle informazioni di debug

```

# Analisi info di debug sul dataset di training
print("Numero di classi nel dataset di training:", len(train_dict))
print("Classi nel dataset di training:", ", ".join(train_dict.keys()))
print("Tipi di dati caricati tramite librosa:")
print(f"Tipo di dato della tupla (audio, sample rate): {type(train_dict['dog'][0])}")
print(f"Tipo di dato dell'audio: {type(train_dict['dog'][0][0])}")
print(f"Tipo di dato del sample rate: {type(train_dict['dog'][0][1])}")

```

Creazione del DataFrame per i dati di debug

```

data = []
for label, samples in train_dict.items():
    for audio, sr in samples:
        duration = len(audio) / sr
        data.append([label, duration, sr])

```

```

df = pd.DataFrame(data, columns=["Label", "Duration", "SampleRate"])

```

Creazione del subplot 3x3

```

fig, axes = plt.subplots(3, 3, figsize=(15, 10))
fig.suptitle('Statistiche dei campioni audio', fontsize=16)

```

1. Numero di campioni per label

```

sns.countplot(data=df, x="Label", ax=axes[0, 0])
axes[0, 0].set_title('Numero di campioni per label')
axes[0, 0].set_ylabel('Numero di campioni')
axes[0, 0].tick_params(axis='x')
for p in axes[0, 0].patches:
    axes[0, 0].annotate(f'{int(p.get_height())}', (p.get_x() + p.get_width() / 2,
                                                    p.get_y() + 1),
                        ha='center', va='bottom', fontsize=10, color='black')

```

2. Distribuzione della durata degli audio

```

sns.histplot(df["Duration"], bins=30, ax=axes[0, 1], color="salmon")
axes[0, 1].set_title('Distribuzione della durata degli audio')

```

```
axes[0, 1].set_ylabel('Numero di audio')

# 3. Distribuzione del sample rate
sns.histplot(df["SampleRate"], bins=30, ax=axes[0, 2], color="lightgreen")
axes[0, 2].set_title('Distribuzione del sample rate')
axes[0, 2].set_ylabel('Numero di audio')
axes[0, 2].set_xticks(sorted(df["SampleRate"].unique()))

# Seconda riga: Distribuzione della durata per ogni label
labels = df["Label"].unique()
num_labels = len(labels)

for i, label in enumerate(labels):
    row = 1 # Riga dei grafici della durata
    col = i
    sns.histplot(df[df["Label"] == label]["Duration"], bins=30, ax=axes[row, col])
    axes[row, col].set_title(f'Durata degli audio - {label}')
    axes[row, col].set_ylabel('Numero di audio')

# Terza riga: Distribuzione del sample rate per ogni label
for i, label in enumerate(labels):
    row = 2 # Riga dei grafici del sample rate
    col = i
    sample_rate_values = df[df["Label"] == label]["SampleRate"].unique()

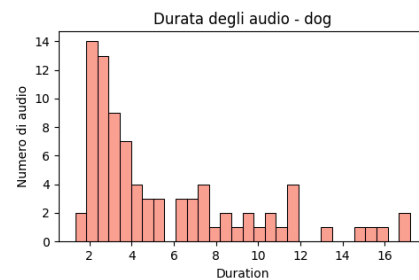
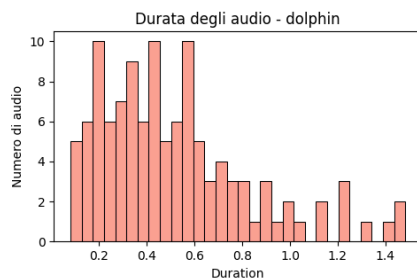
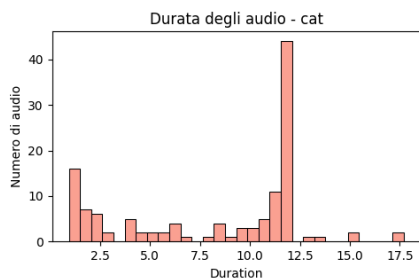
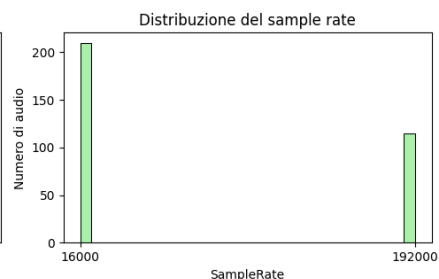
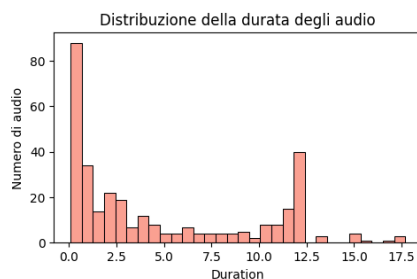
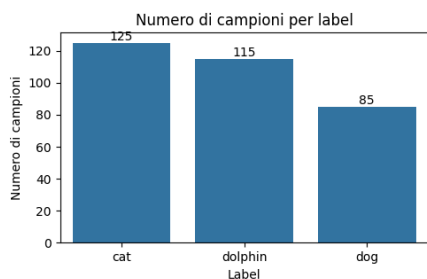
    # Se c'è un solo valore di sample rate, usiamo un grafico a torta
    if len(sample_rate_values) == 1:
        axes[row, col].pie([1], labels=[f'Sample Rate: {sample_rate_values[0]}'],
        axes[row, col].set_title(f'Sample Rate - {label}')
    else:
        # Se ci sono più valori, usiamo un istogramma
        sns.histplot(df[df["Label"] == label]["SampleRate"], bins=30, ax=axes[row, col])
        axes[row, col].set_title(f'Sample Rate - {label}')
        axes[row, col].set_ylabel('Numero di audio')

# Ottimizzazione del layout
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```



```
Numero di classi nel dataset di training: 3
Classi nel dataset di training: cat, dolphin, dog
Tipi di dati caricati tramite librosa:
Tipo di train_dict['dog'][0]: <class 'tuple'>
Tipo di train_dict['dog'][0][0]: <class 'numpy.ndarray'>
Tipo di train_dict['dog'][0][1]: <class 'int'>
```

Statistiche dei campioni audio



Sample Rate - cat

Sample Rate - dolphin

Sample Rate - dog

Sample Rate: 16000

100.0%

Sample Rate: 192000

100.0%

Sample Rate: 16000

100.0%

✓ 3.3. Analisi dei grafici e considerazioni sui dati

Dall'osservazione dei grafici emerge quanto segue:

- **Numero di campioni disponibili**

Il dataset è composto da **325 campioni totali**, un numero probabilmente insufficiente per l'addestramento di reti neurali troppo complesse.

- **Lunghezza variabile dei campioni e implicazioni computazionali**

La lunghezza dei campioni varia, con il più lungo pari a **17,5 secondi**. Considerando una frequenza di campionamento di **16 kHz**, questo corrisponde a:

$$16000 \text{ Hz} \times 17,5 \text{ s} = 280000 \text{ campioni}$$

Tale dimensione dell'input risulta proibitiva in termini di gestione computazionale. A questo scopo risulta indispensabile un **Troncamento** dei campioni.

- **Frequenza di campionamento**

I campioni audio sono distribuiti su due diverse frequenze di campionamento:

- Gli audio di **cani e gatti** sono stati campionati a **16 kHz**.
- Gli audio di **delfini** sono stati campionati a **192 kHz**.

Questo risultato è coerente con l'analisi teorica svolta in precedenza, poiché i versi dei delfini vengono emessi a frequenze significativamente più elevate rispetto a quelle di cani e gatti. In particolare, secondo il **Teorema di Nyquist**:

Un segnale periodico deve essere campionato a più del doppio della componente di frequenza più alta del segnale.

Ne consegue che, se la frequenza di campionamento fosse troppo bassa, i versi dei delfini potrebbero non essere adeguatamente catturati.

Tuttavia, non è possibile utilizzare per l'addestramento due frequenze di campionamento differenti, specialmente in un caso come il nostro, in cui tutti gli audio di ciascuna classe condividono una specifica frequenza di campionamento. Durante la fase di inferenza, non avendo il valore target, sarà necessario effettuare il **resampling** a una frequenza di campionamento uniforme.

La **frequenza di campionamento** è definita come:

Il numero di campioni presenti in un secondo di segnale digitale, espresso in Hertz.

Utilizzare frequenze di campionamento differenti porterebbe alla perdita di una delle feature fondamentali individuate nell'analisi teorica, ovvero la **dipendenza temporale e la**

cadenza dei versi. Infatti, un audio con una frequenza di campionamento più alta rispetto ai dati di training sarebbe interpretato dal modello come rallentato, mentre uno con una frequenza inferiore risulterebbe velocizzato.

Definiamo delle funzioni per stampare nel notebook le forme d'onda, gli spettrogrammi e ascoltare un audio.

```
# Riproduce audio direttamente ne Notebook
def play_audio(audio, sr):
    ipd.display(ipd.Audio(audio, rate=sr))

# Mostra la forma d'onda
def plot_waveform(audio, sr, fig, ax, title='Forma d\'onda'):
    librosa.display.waveshow(audio, sr=sr, ax=ax)
    ax.set_title(title)
    ax.set_xlabel('Tempo (s)')
    ax.set_ylabel('Ampiezza')

# Mostra lo spettrogramma STFT
def plot_spectrogram_STFT(audio, sr, fig, ax, y_scale='log', title='Spettrogramma
D = librosa.stft(audio)
S_db = librosa.amplitude_to_db(np.abs(D), ref=np.max)

img = librosa.display.specshow(S_db, sr=sr, x_axis='time', y_axis=y_scale, ax

ax.set_title(title)
fig.colorbar(img, ax=ax, format='%+2.0f dB')
ax.set_ylabel('Frequenza (Hz)')
ax.set_xlabel('Tempo (s)')

# Mostra lo spettrogramma Mel
def plot_spectrogram_Mel(audio, sr, fig, ax, nmels=128, title='Spettrogramma (Mel
S = librosa.feature.melspectrogram(y=audio, sr=sr, n_mels=nmels)
S_db_mel = librosa.amplitude_to_db(S, ref=np.max)

img = librosa.display.specshow(S_db_mel, sr=sr, x_axis='time', y_axis='mel',

ax.set_title(title)
fig.colorbar(img, ax=ax, format='%+2.0f dB')
ax.set_ylabel('Frequenza (Hz)')
ax.set_xlabel('Tempo (s)')
```

Stampiamo per dimostrazioni un esempio di forma d'onda, spettrogramma e relativo audio per ogni classe.

```
print("\033[1;34m Esempio di audio - Delfino:\033[0m")
play_audio(train_dict['dolphin'][0][0], train_dict['dolphin'][0][1])
fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(15, 4))
plot_waveform(train_dict['dolphin'][0][0], train_dict['dolphin'][0][1], fig, ax1)
plot_spectrogram_STFT(train_dict['dolphin'][0][0], train_dict['dolphin'][0][1], f
```

```
plt.tight_layout()
plt.show()
```

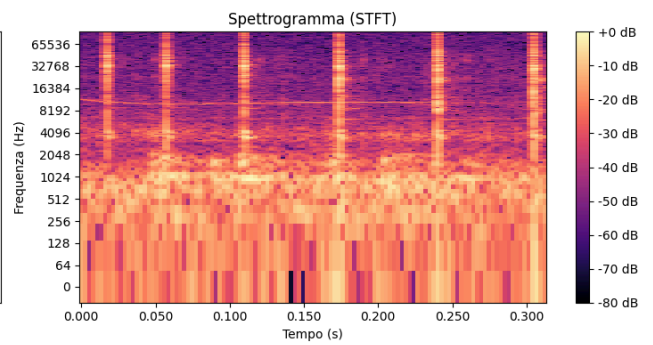
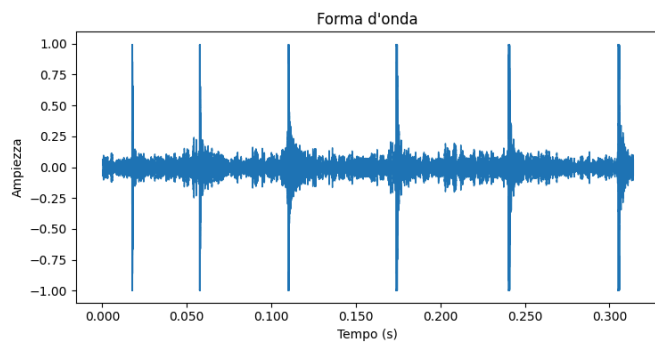
```
print("\033[1;34m Esempio di audio - Gatto:\033[0m")
play_audio(train_dict['cat'][0][0], train_dict['cat'][0][1])
fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(15, 4))
plot_waveform(train_dict['cat'][0][0], train_dict['cat'][0][1], fig, ax1)
plot_spectrogram_STFT(train_dict['cat'][0][0], train_dict['cat'][0][1], fig, ax2)
plt.tight_layout()
plt.show()
```

```
print("\033[1;34m Esempio di audio - Cane:\033[0m")
play_audio(train_dict['dog'][0][0], train_dict['dog'][0][1])
fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(15, 4))
plot_waveform(train_dict['dog'][0][0], train_dict['dog'][0][1], fig, ax1)
plot_spectrogram_STFT(train_dict['dog'][0][0], train_dict['dog'][0][1], fig, ax2)
plt.tight_layout()
plt.show()
```



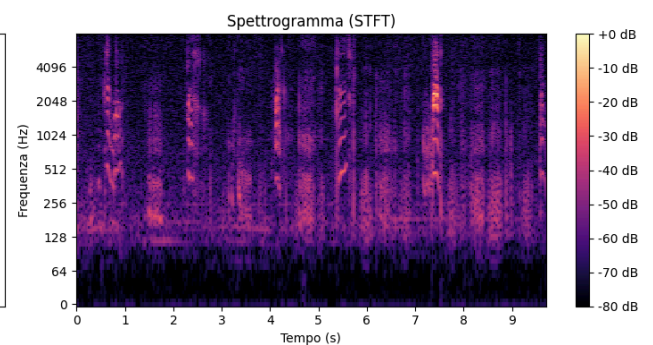
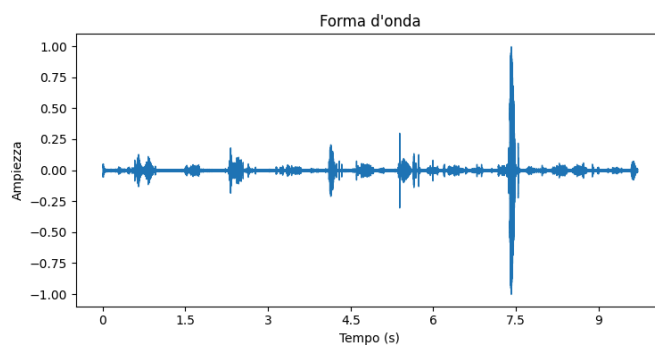
Esempio di audio - Delfino:

0:00 / 0:00



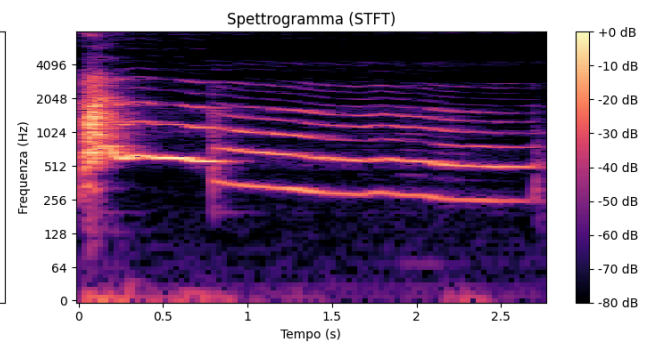
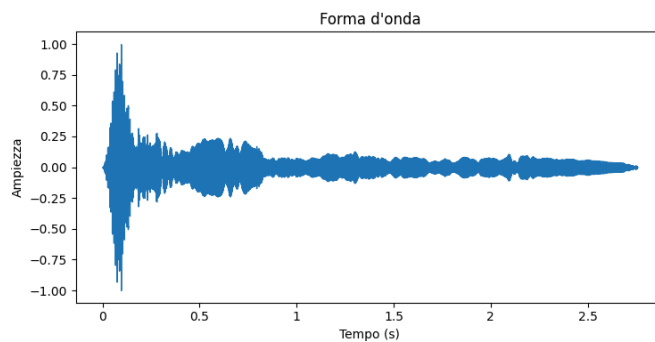
Esempio di audio - Gatto:

0:00 / 0:09



Esempio di audio - Cane:

0:00 / 0:02



✓ 3.4 Data augmentation

Per incrementare il numero di campioni di training, possiamo applicare diverse tecniche di **data augmentation**, suddivise in base al tipo di trasformazione.

1. Trasformazioni temporali

- **Time Stretching** ✗

Alterare la velocità di emissione del verso potrebbe generare dati irrealistici, poiché la velocità è una caratteristica distintiva di ogni specie.

- **Pitch Shifting** ✗

Modificare la tonalità di un verso non è appropriato, in quanto la frequenza è una feature fondamentale e un'alterazione porterebbe alla perdita di informazioni importanti, influenzando anche i processi di riduzione del rumore.

- **Time Shifting** ✓

Spostare temporalmente il segnale senza modificarne il contenuto potrebbe essere utile, in quanto non altera la natura del verso ma rende il modello più robusto rispetto alla posizione dell'audio nel tempo.

- **Time Masking** ✓

Simile al **dropout** sui dati di training, questa tecnica oscura parti dell'audio, favorendo una maggiore generalizzazione del modello.

2. Trasformazioni basate sulla frequenza

- **Spec Augment** ✓

Analogo al **Time Masking**, ma operante sul dominio della frequenza. Oscurare selettivamente parti dello spettrogramma aiuta a rendere il modello più robusto a variazioni nelle frequenze emesse dagli animali.

3. Trasformazioni basate sul rumore

- **Aggiunta di rumore ambientale** ✓

Integrare rumori di fondo realistici o riverberi migliora la capacità del modello di generalizzare a contesti reali.

- **Volume Scaling** ✓

Modificare l'intensità dell'audio consente di variare la dinamica del segnale, simulando registrazioni effettuate con diverse condizioni di acquisizione.

4. Segmentazione e permutazione dei versi

Durante lo sviluppo del progetto, è emersa un'idea innovativa per aumentare la variabilità dei dati: **segmentare i versi degli animali all'interno di ciascuna traccia audio ed effettuare una permutazione di questi segmenti**. L'idea e il risultato sono particolarmente promettenti poiché permette di sintetizzare un numero fattoriale di nuovi audio che cercano di imitare il continuo delle registrazioni originali. Il problema nasce dalle risorse computazionali utilizzate per lo sviluppo del progetto. Anche con l'utilizzo della più performante GPU offerta da Google Colab *NVIDIA L4* il training di sequenze così lunghe di dati richiede una grande quantità di tempo. Per questo motivo, la sperimentazione di questa tecnica è stata spostata alla parte finale del notebook, come possibile strategia alternativa per future ottimizzazioni del dataset o come idea per problemi più complessi.

5. Segmentazione del dataset

Data la dimensione ridotta del training set, è stato possibile analizzare manualmente tutti i campioni con una lunghezza superiore a **3 secondi**. Questa analisi empirica ha confermato che segmentando gli audio più lunghi in porzioni di **3 secondi**, ogni segmento conterrà **almeno un verso dell'animale**.

(questa operazione è attuabile solo ed esclusivamente perchè i dati di training sono stati analizzati singolarmente per confermare la precedente affermazione)

Una dimostrazione di questa osservazione è disponibile nel capitolo finale [Segmentazione e Permutazione dei Versi](#), dove viene illustrato il campionamento dei silenzi tra le vocalizzazioni all'interno dello stesso audio.

Possiamo sfruttare questa proprietà per aumentare la dimensione del nostro dataset, senza introdurre alterazioni artificiali nei dati.

Si è scelto di segmentare gli audio in blocchi di **3 secondi**, anche se le pause tra le vocalizzazioni sono generalmente più brevi, per i seguenti motivi:

- **Preservare la cadenza delle vocalizzazioni**, una delle caratteristiche chiave identificate nell'[Analisi Teorica](#).
- **Garantire una lunghezza uniforme degli input**, semplificando il processamento e il training del modello.
- **Evitare il troncamento di informazioni rilevanti**, assicurando che ogni segmento mantenga la struttura originale del verso.

- **Compromesso ottimale** per l'efficienza computazionale.

Questa scelta consente di **migliorare la qualità dei dati di training**, offrendo al modello una rappresentazione più realistica della variabilità nei versi degli animali.

Definiamo una funzione che suddivide gli audio con durata superiore a 3 secondi in segmenti da 3 secondi ciascuno.

Ogni audio viene diviso in blocchi di 3 secondi. Se la lunghezza dell'audio non è un multiplo di 3 secondi, la parte rimanente viene gestita in base alla sua durata:

- Se il segmento residuo è più corto del valore impostato in `min_duration`, viene scartato per evitare di introdurre campioni contenenti solo rumore.
- Se supera `min_duration`, viene mantenuto come ulteriore campione.

Questa strategia aiuta a evitare campioni troppo brevi e garantisce coerenza negli input del modello, preservando al contempo la cadenza delle vocalizzazioni analizzata in precedenza. La funzione verrà applicata a tutti gli audio del dataset per uniformarne la durata prima della fase di training.

```
# Funzione che divide gli audio in segmenti da una lunghezza prefissata e restituisce
def split_audio_to_segments(audio_tuple, segment_duration=3, min_duration=2):
    audio_data, sample_rate = audio_tuple # Estrai l'array audio e il sample rate
    samples_per_segment = segment_duration * sample_rate # Calcola il numero di campioni per segmento
    segments = [] # Lista per i segmenti
    # Se l'audio è più corto della durata richiesta, restituisco così com'è
    if len(audio_data) < samples_per_segment:
        segments.append((audio_data, sample_rate))
        return segments
    # Se l'audio è più lungo, suddividilo in segmenti completi
    for start in range(0, len(audio_data), samples_per_segment):
        end = start + samples_per_segment
        segment = audio_data[start:end]
        # Aggiungi il segmento solo se è lungo almeno `min_duration`
        if len(segment) >= min_duration*sample_rate:
            segments.append((segment, sample_rate))

    return segments
```

Allochiamo un nuovo dizionario che conterrà tutti gli audio originali segmentati.

```
# Dizionario degli audio segmentati
segmented_train_dict = defaultdict(list)

# Segmentiamo gli audio originali in parti da 3 secondi e le inseriamo nel nuovo dizionario
for label, samples in train_dict.items():
    for sample in samples:
        segments = split_audio_to_segments(sample, 3, 2)
        segmented_train_dict[label].extend(segments)
```

```
#Stampiamo informazioni di debug per sapere il nuovo numero di campioni di triani
print("Numero di campioni per classe nel dataset di training segmentato:")
print(f"- Classe 'dog': {len(segmented_train_dict['dog'])} campioni")
print(f"- Classe 'cat': {len(segmented_train_dict['cat'])} campioni")
print(f"- Classe 'dolphin': {len(segmented_train_dict['dolphin'])} campioni")
```

```
➡ Numero di campioni per classe nel dataset di training segmentato:
- Classe 'dog': 155 campioni
- Classe 'cat': 355 campioni
- Classe 'dolphin': 115 campioni
```

Come era prevedibile dall'analisi precedente del training set, il numero dei campioni della classe **gatti** si è notevolmente aumentato, essendo quella con gli audio più lunghi. Il numero degli audio dei **cani** è anch'esso cresciuto, mentre quello dei **delfini** è rimasto invariato, dato che tutti i loro audio erano inferiori ai 3 secondi.

Per evitare di avere un dataset sbilanciato, dobbiamo applicare alcune delle tecniche di **dataset augmentation** precedentemente descritte, in particolare per le classi dei cani e dei delfini. Una delle tecniche che utilizzeremo è il **volume scaling**, che permette di rendere i modelli più robusti ai cambiamenti nel volume dell'audio. Questa è una tecnica appropriata, poiché la variazione di volume dipende dal **microfono** usato per la registrazione, e non è una caratteristica intrinseca della vocalizzazione degli animali.

Inoltre, per evitare che lo scaling produca valori fuori dall'intervallo normale di **[-1, 1]** (valore tipico per i dati audio normalizzati), utilizziamo la funzione `np.clip` di numpy. Questa funzione modifica le forme d'onda **appiattend** i picchi che escono dall'intervallo, garantendo che i valori rientrino nella gamma accettabile senza distorcere eccessivamente il segnale. Questo può essere visto per i modelli che si baseranno sulle forme d'onda grezze come un rumore aggiuntivo visto che si perde una parte della curva nei picchi di intensità.

```
# Volume scaling
```

```
# Numero di audio sintetici per ogni audio
```

```
NUM_AUG_VOLUME_DOG = 1
```

```
NUM_AUG_VOLUME_CAT = 0
```

```
NUM_AUG_VOLUME_DOLPHIN = 2
```

```
# Funzione che effettua il volume scaling su un segnale audio
```

```
def volume_scaling(audio_tuple, scaling_range=(0.5, 1.5)):
```

```
    audio_data, sample_rate = audio_tuple
```

```
    # Genera un fattore casuale tra scaling_range[0] e scaling_range[1]
```

```
    scale_factor = np.random.uniform(scaling_range[0], scaling_range[1])
```

```
    # Applica il volume scaling
```

```
    scaled_audio = audio_data * scale_factor
```

```
    # Clipping per evitare valori fuori dall'intervallo [-1, 1] (se normalizzato)
```

```
    scaled_audio = np.clip(scaled_audio, -1.0, 1.0)
```

```
    return scaled_audio
```

```
# Applichiamo volume scaling ai segmenti campione
augmented_train_dict = defaultdict(list)

for label, samples in segmented_train_dict.items():
    if label == 'dog':
        for sample in samples:
            for _ in range(NUM_AUG_VOLUME_DOG):
                augmented_train_dict[label].append((volume_scaling(sample), sample[1]))
    if label == 'cat':
        for sample in samples:
            for _ in range(NUM_AUG_VOLUME_CAT):
                augmented_train_dict[label].append((volume_scaling(sample), sample[1]))
    if label == 'dolphin':
        for sample in samples:
            for _ in range(NUM_AUG_VOLUME_DOLPHIN):
                augmented_train_dict[label].append((volume_scaling(sample), sample[1]))

print(len(augmented_train_dict['dog']))
print(len(augmented_train_dict['cat']))
print(len(augmented_train_dict['dolphin']))
```

```
⇒ 155
   0
   230
```

```
print("totale audio di training per gatti:", len(augmented_train_dict['cat'])+len(
print("totale audio di training per cani:", len(augmented_train_dict['dog'])+len(
print("totale audio di training per dolini:", len(augmented_train_dict['dolphin']
```

```
⇒ totale audio di training per gatti: 355
   totale audio di training per cani: 310
   totale audio di training per dolini: 345
```

Il numero di campioni totali di training dopo la data augmentation è diventato 1010

✓ 3.5 Resampling

Per quanto riguarda la frequenza di campionamento, come detto precedentemente, è necessario scegliere una frequenza uniforme per tutti gli audio. L'obiettivo è trovare un trade-off tra la perdita di informazioni dovuta al downscaling e la grandezza delle sequenze di input derivanti dalla frequenza di campionamento.

La scelta è stata fatta a **16 kHz**, poiché questa è una frequenza di campionamento standard che minimizza la quantità di dati per secondo di audio, a discapito della qualità degli audio dei delfini.

In questo modo, gli audio che utilizzeremo avranno una durata di **3 secondi** e una frequenza di campionamento di **16 kHz**, con una dimensione complessiva di

$$3 \text{ s} \cdot 16000 \text{ Hz} = 48000 \text{ campioni per segmento}$$

Definiamo una funzione che effettua il **resampling** degli audio nel caso in cui la loro frequenza di campionamento sia diversa da quella scelta (16 kHz). La funzione prenderà in input i campioni audio, eseguirà il resampling alla frequenza di campionamento desiderata e salverà i file risultanti all'interno di una cartella mantenendo la stessa strutturazione del dataset:

```
main_directory/
...class_a/
.....a_audio_1.wav
.....a_audio_2.wav
...class_b/
.....b_audio_1.wav
.....b_audio_2.wav

# Funzione che salva il file in una cartella di destinazione seguendo lo standard
def resample_and_save_from_dict(audio_dict, desired_sr, main_folder):
    # Itera attraverso il dizionario con le label come chiavi
    for label, samples in audio_dict.items():
        # Crea la cartella per la label se non esiste
        label_folder = os.path.join(main_folder, label)
        os.makedirs(label_folder, exist_ok=True)
        for idx, (audio, original_sr) in enumerate(samples):
            # Esegui il resampling
            audio_resampled = librosa.resample(audio, orig_sr=original_sr, target_sr=
            # Crea il nome del file con l'indice (esempio: label_0_16000Hz.wav)
            filename = f"{label}_{idx}_{desired_sr}Hz.wav"
            # Crea il percorso completo dove salvare il file
            output_path = os.path.join(label_folder, filename)
            # Salva il file audio resampled
            sf.write(output_path, audio_resampled, desired_sr)
```

Impostiamo il target Sample Rate a 16000 Hz e effettuiamo il resampling sia del training set originale sia dei dati sintetizzati

```
TARGET_SR = 16000 # Sample rate di riferimento
SINTETIC_TRAIN_PATH = "/content/drive/MyDrive/Audio_Animal_Recognition/sintetic_t
SEGMENTED_ORIGINAL_TRAIN_PATH = "/content/drive/MyDrive/Audio_Animal_Recognition/

resample_and_save_from_dict(augmented_train_dict, TARGET_SR, SINTETIC_TRAIN_PATH)
resample_and_save_from_dict(segmented_train_dict, TARGET_SR, SEGMENTED_ORIGINAL_T
```

Il **resampling** di un audio richiede una grande quantità di **memoria RAM**. Per ridurre il carico computazionale durante le fasi di inferenza, è preferibile effettuare il **resampling** anche sui dati

di **test** prima di procedere con l'addestramento del modello.

Questo approccio ha il vantaggio di evitare il resampling in tempo reale durante il processo di inferenza, riducendo così il tempo di esecuzione e la richiesta di memoria.

Di seguito, possiamo applicare la stessa logica di resampling usata per i dati di training anche ai dati di test, utilizzando la funzione definita precedentemente:

```
ORIGINAL_TEST_PATH = "/content/drive/MyDrive/Dataset/cats_dogs_dolphins/test" # d

# Controllo esistenza directory
if not os.path.exists(ORIGINAL_TEST_PATH):
    print(f"Attenzione: la directory {ORIGINAL_TEST_PATH} non esiste!")

TEST_PATH = "/content/drive/MyDrive/Audio_Animal_Recognition/segmented_test" # di

original_test_dict = defaultdict(list)
load_from_directory(ORIGINAL_TEST_PATH, original_test_dict)
# qui non effettuiamo naturalmente la segmentazione dei dati
resample_and_save_from_dict(original_test_dict, TARGET_SR, TEST_PATH)
```

✓ 4. Modelli

I modelli che andremo ad analizzare si suddividono in due categorie principali, in base al tipo di input che ricevono.

Modelli basati su forme d'onda grezze

Per i modelli che riceveranno in ingresso le forme d'onda grezze, le aspettative sui risultati non sono molto elevate. La ragione principale è che la forma d'onda grezza non fornisce alcuna informazione esplicita sulla **frequenza** del suono, una caratteristica fondamentale per la classificazione dei versi degli animali. Inoltre, come osservato durante la fase di esplorazione del dataset, l'input shape per questi modelli è di (1, 48000), il che significa che il modello dovrà gestire una grande quantità di dati per ogni campione. Questo inevitabilmente si tradurrà in un numero di parametri molto elevato, che può aumentare la complessità del modello e il tempo di addestramento. Per questi modelli infatti sarà richiesta una **GPU** per l'addestramento in tempi efficienti.

In questa categoria andremo ad esplorare un modello convoluzionale **CNN** che si ispira alla rete **WaveNet** vista a lezione. La rete WaveNet è una DNN che nasce con lo scopo di generazione di audio. Cercheremo di adattarla e semplificarla per la classificazione dell'audio. È il punto di partenza ideale visto che nasce ricevendo in input le forme d'onda grezze.

Modelli basati su spettrogrammi

La seconda categoria comprende i modelli che riceveranno in ingresso gli **spettrogrammi** degli audio. Poiché gli spettrogrammi forniscono una rappresentazione visiva delle frequenze

contenute nel segnale audio, l'input shape in questo caso sarà significativamente più ridotto rispetto alle forme d'onda grezze. Di conseguenza, questi modelli avranno un numero inferiore di parametri da apprendere, il che potrebbe renderli più efficienti dal punto di vista computazionale.

In questa categoria, si inizierà con un modello di rete neurale convoluzionale di base come **AlexNet**, per poi esplorare miglioramenti successivi.

Inoltre, come accennato durante l'analisi teorica, una **frequenza di campionamento maggiore** potrebbe portare a migliori risultati, specialmente nella classificazione dei versi dei **delfini**.

✓ 4.1. Import utilizzati

Gli import utilizzati in questo capitolo sono:

- **tensorflow**: utilizzato per la creazione dei modelli e per il caricamento del dataset.
- **numpy**: utilizzato per la manipolazione degli array numpy.
- **sklearn.metrics**: utilizzato per la valutazione dei modelli.
- **seaborn e matplotlib**: utilizzati per stampare i grafici di valutazione.
- **defaultdict, Counter**: utilizzati per lo split del validation set.
- **librosa**: utilizzato per la stampa sul notebook degli spettrogrammi e delle forme d'onda.

```
import tensorflow as tf
import numpy as np
from tensorflow.keras import layers, models
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import precision_score, recall_score, f1_score, confusion_ma
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
from collections import defaultdict, Counter
import librosa
```

✓ 4.2. Caricamento del Dataset

In questo secondo capitolo del notebook si ricaricano i dataset pre-elaborati nel capitolo precedente, in modo che questo secondo capitolo possa essere eseguito indipendentemente dal primo. Questo è particolarmente utile in Google Colab, dove è possibile cambiare il runtime e quindi azzerare la RAM. A questo scopo, saranno ridefinite alcune variabili globali.

```

TEST_PATH = "/content/drive/MyDrive/Audio_Animal_Recognition/segmented_test" # di
SINTETIC_TRAIN_PATH = "/content/drive/MyDrive/Audio_Animal_Recognition/sintetic_t
SEGMENTED_ORIGINAL_TRAIN_PATH = "/content/drive/MyDrive/Audio_Animal_Recognition/
TARGET_SR = 16000 # sample rate congruo a quello scelto nel capitolo precedente
MAX_DURATION = 3 #secondi durata dei campioni congrua a quella scelta nel capitol

```

Definiamo una funzione che utilizza `tf.keras.utils.audio_dataset_from_directory`.

Questa funzione consente di importare i dati in un oggetto `tf.data.Dataset`, implementando in un'unica operazione le seguenti trasformazioni:

- **One-hot encoding** delle etichette
- **Padding** degli audio con durata inferiore alla soglia predefinita
- **Shuffling** dei dati per garantire una migliore distribuzione dei campioni

Questa soluzione permette di semplificare il pre-processing e rendere più leggibile il codice.

```

# Funzione che data una directory restituisce il dataset
def load_audio_data_tensorflow(directory, length=TARGET_SR*MAX_DURATION, shuffle=
    dataset = tf.keras.utils.audio_dataset_from_directory(
        directory,
        labels='inferred',
        label_mode='categorical',
        batch_size=None,
        shuffle=shuffle,
        seed=seed,
        output_sequence_length=length, # Lunghezza fissa delle forme d'onda
    )
    return dataset

```

Utilizziamo la funzione appena definita per caricare in memoria i dataset dei dati sintetici, originali e di test.

```

sintetic_train_ds = load_audio_data_tensorflow(SINTETIC_TRAIN_PATH)
original_train_ds = load_audio_data_tensorflow(SEGMENTED_ORIGINAL_TRAIN_PATH)
test_ds = load_audio_data_tensorflow(TEST_PATH)

```

```

➡ Found 385 files belonging to 3 classes.
   Found 625 files belonging to 3 classes.
   Found 98 files belonging to 3 classes.

```

Per debug e ci tornerà utile nella stampa dei grafici allochiamo in memoria una variabile `class_names` che contiene una lista dei target caricati dal dataset.

```

class_names = original_train_ds.class_names
print(f"Classi trovate: {class_names}")

```

```

➡ Classi trovate: ['cat', 'dog', 'dolphin']

```

Per comprendere meglio come modellare i dati prima di fornirli ai modelli, stampiamo ulteriori informazioni di debug.

```
print(type(sintetic_train_ds)) # che tipo di dato è sintetic_train_ds
print(sintetic_train_ds.element_spec) # che tipologie di dati contiene
print(sintetic_train_ds.cardinality()) # cardinalità di sintetic_train_ds
print(original_train_ds.cardinality()) # cardinalità di original_train_ds
print(test_ds.cardinality()) # cardinalità di test_ds
```

```
>>> <class 'tensorflow.python.data.ops.prefetch_op._PrefetchDataset'>
      (TensorSpec(shape=(48000, None), dtype=tf.float32, name=None), TensorSpec(shape=(48000, None), dtype=tf.float32, name=None),
      <_TakeDataset element_spec=(TensorSpec(shape=(48000, None), dtype=tf.float32, name=None),
      tf.Tensor(385, shape=(), dtype=int64),
      tf.Tensor(625, shape=(), dtype=int64),
      tf.Tensor(98, shape=(), dtype=int64))
```

✓ 4.3. Suddivisione del Training Set e Pre-processing

A questo punto, suddividiamo il training set, composto sia dai dati originali di training che da quelli sintetizzati, in due insiemi: **training** e **validation**. Adottiamo una suddivisione standard, assegnando il **15% dei dati alla validazione**, selezionandoli esclusivamente dall'insieme dei dati originali e non sintetizzati.

Inoltre, applichiamo ulteriori trasformazioni ai dati:

- **Reshaping** per garantire la compatibilità con i modelli che utilizzeremo
- **Suddivisione in batch** per ottimizzare il processo di addestramento

```
VALID_PERCENT = 0.15
```

```
num_validation = int(VALID_PERCENT * (original_train_ds.cardinality().numpy() + sintetic_train_ds.cardinality().numpy()))
print(num_validation)
```

```
>>> 151
```

```
def reshape_audio(audio, label):
    # Ridimensiona l'audio a forma (x,)
    audio = tf.squeeze(audio, axis=-1)
    return audio, label
```

```
# Preprocessing dei dataset
original_train_ds = original_train_ds.shuffle(1000).map(reshape_audio)
sintetic_train_ds = sintetic_train_ds.shuffle(1000).map(reshape_audio)
test_ds = test_ds.shuffle(100).map(reshape_audio)
```

```
# Raggruppa gli esempi per classe
class_groups = defaultdict(list)
```

```
# Itera su tutti gli esempi nel dataset `original_train_ds` per raccogliere i dati
```

```

for audio, label in original_train_ds:
    label = label.numpy() # Converti il tensor a numpy per manipolarlo
    class_idx = np.argmax(label) # Trova l'indice del valore 1 nel one-hot encod
    class_groups[class_idx].append((audio, label)) # Aggiungi l'esempio al grupp

# Calcola il numero di esempi per classe:
num_valid_class = num_validation // len(class_names)

# Passo 3: Seleziona un numero uguale di campioni per ciascuna classe per il vali
validation_samples = []
for class_idx, group in class_groups.items():
    # Seleziona i primi `min_class_size` campioni da ciascun gruppo
    validation_samples.extend(group[:num_valid_class])

# Converte audio e label separatamente per garantire che le forme siano compatibi
validation_audio = [sample[0] for sample in validation_samples]
validation_labels = [sample[1] for sample in validation_samples]

validation_ds = tf.data.Dataset.from_tensor_slices((validation_audio, validation_

# Usa gli esempi rimanenti per il training
train_samples = []
for class_idx, group in class_groups.items():
    train_samples.extend(group[num_valid_class:]) # Aggiunge gli esempi rimanent

# Crea il dataset di addestramento
train_audio = [sample[0] for sample in train_samples]
train_labels = [sample[1] for sample in train_samples]

# Crea il dataset di addestramento
train_ds = tf.data.Dataset.from_tensor_slices((train_audio, train_labels)).concat

# Crea un contatore per le etichette
label_counter = Counter()

# Itera attraverso il dataset e conta le etichette
for _, label in validation_ds:
    # Trova l'indice dell'elemento 1 nel vettore one-hot per ottenere la classe
    class_idx = np.argmax(label.numpy())
    label_counter[class_idx] += 1

# Stampa la frequenza di ciascuna classe
print("Frequenza di ciascuna classe nel validation set:")
for label, count in label_counter.items():
    print(f"Classe {class_names[label]}: {count} campioni")

train_ds = train_ds.batch(32)
validation_ds = validation_ds.batch(32)
test_ds = test_ds.batch(32)

print("-----")
print("Shape dei dataset:")
print(train_ds.element_spec)
print(validation_ds.element_spec)
print(test_ds.element_spec)

```

```

↩ Frequenza di ciascuna classe nel validation set:
Classe cat: 50 campioni
Classe dolphin: 50 campioni
Classe dog: 50 campioni
-----
Shape dei dataset:
(TensorSpec(shape=(None, 48000), dtype=tf.float32, name=None), TensorSpec(shape=(None, 48000), dtype=tf.float32, name=None), TensorSpec(shape=(None, 48000), dtype=tf.float32, name=None), TensorSpec(shape=(None, 48000), dtype=tf.float32, name=None), TensorSpec(shape=(None, 48000), dtype=tf.float32, name=None), TensorSpec(shape=(None, 48000), dtype=tf.float32, name=None))

```

✓ 4.4. Definizione delle Funzioni di Addestramento e Valutazione

Per migliorare la leggibilità del codice e strutturare il flusso di lavoro in modo chiaro ed efficiente, definiamo due funzioni principali:

1. `compile_and_fit`

Questa funzione consente di:

- **Compilare** il modello
- **Addestrare** il modello sul dataset di training
- **Valutare** le prestazioni del modello durante l'addestramento
- **Generare i grafici** di **loss** e **accuracy** per il training set e il validation set

Questi grafici permettono di monitorare l'andamento del modello, individuare eventuali fenomeni di **overfitting** e valutare il miglior modello ottenuto che sarà quello scelto per la valutazione finale sul test set.

2. Funzione di valutazione sui dati di test

La seconda funzione è dedicata alla **valutazione finale** del modello migliore selezionato precedentemente sui dati di validation. In particolare, calcola e analizza:

- **Matrice di confusione**: utile per visualizzare le prestazioni del modello nella classificazione delle diverse classi
- **Metriche di valutazione**:
 - **Precision**: misura la proporzione di istanze classificate correttamente su tutte quelle classificate come positive
 - **Recall**: misura la capacità del modello di identificare correttamente tutte le istanze positive
 - **F1-score**: metrica che bilancia precision e recall

Questa funzione fornisce una valutazione oggettiva delle prestazioni del modello che potranno essere fornite a chi utilizzerà il modello nella fase di inferenza.

```

# Compila un modello, stampa il sommario, addestra il modello e stampa i valori
def compile_and_fit(model, train_ds, validation_ds, epochs, optimizer, batch_size):
    # Compila il modello
    model.compile(optimizer=optimizer,

```

```

        loss='categorical_crossentropy',
        metrics=['accuracy'])
# Stampa il sommario dei livelli e dei parametri
model.summary()
# Addestra il modello
history = model.fit(train_ds,
                    validation_data=validation_ds,
                    epochs=epochs,
                    batch_size=batch_size,
                    callbacks=callbacks)

# Stampiamo i valori finali di training loss, training accuracy, validation l
best_epoch = history.history['val_loss'].index(min(history.history['val_loss']
print(f"Training loss: {history.history['loss'][best_epoch]}")
print(f"Training accuracy: {history.history['accuracy'][best_epoch]}")
print(f"Validation loss: {history.history['val_loss'][best_epoch]}")
print(f"Validation accuracy: {history.history['val_accuracy'][best_epoch]}")

#stampiamo un grafico dell'andamento delle loss e delle accuracy
pd.DataFrame(history.history).plot(figsize=(8, 5))
plt.title("Andamento dell'addestramento modello")
plt.grid(True)
plt.gca().set_ylim(0, 1)
plt.show()

# Funzione che testa il modello su i dati di test, stampa la matrice di confusion
def evaluate_model(model, test_ds):
    # Valutazione del modello sul dataset di test
    test_loss, test_accuracy = model.evaluate(test_ds)
    print(f'Test Loss: {test_loss}')
    print(f'Test Accuracy: {test_accuracy}')

    # Stampa matrice di confusione
    y_true = []
    y_pred = []
    for x_batch, y_batch in test_ds: # test_ds è il dataset di test in batch
        predictions = model.predict(x_batch) # Ottenere predizioni
        y_true.extend(np.argmax(y_batch.numpy(), axis=1)) # Convertire etichette r
        y_pred.extend(np.argmax(predictions, axis=1)) # Convertire predizioni in c
    # Creiamo la matrice di confusione
    conf_matrix = confusion_matrix(y_true, y_pred)
    # Stampiamo la matrice di confusione
    plt.figure(figsize=(8, 6))
    sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues", xticklabels=class
    plt.xlabel("Classe predetta")
    plt.ylabel("Classe vera")
    plt.title("Matrice di confusione")
    plt.show()

#Stampiamo un grafico della precision, recall e f1score
precision = precision_score(y_true, y_pred, average=None)
recall = recall_score(y_true, y_pred, average=None)
f1 = f1_score(y_true, y_pred, average=None)

```



```

print("Metriche per ciascuna classe:")
for i, class_name in enumerate(class_names):
    print(f"{class_name} - Precision: {precision[i]:.4f}, Recall: {recall[i]:.4f}")

x = np.arange(len(class_names)) # Posizioni per le classi
width = 0.2 # Larghezza delle barre

fig, ax = plt.subplots(figsize=(10, 6))
ax.bar(x - width, precision, width, label='Precision', color='blue')
ax.bar(x, recall, width, label='Recall', color='green')
ax.bar(x + width, f1, width, label='F1 Score', color='red')

ax.set_xticks(x)
ax.set_xticklabels(class_names)
ax.set_ylim(0, 1)
ax.set_ylabel('Score')
ax.set_title('Performance del Modello per Classe')
ax.legend()

plt.show()

```

Definiamo in anticipo il meccanismo di **early stopping** e di **decadimento del learning rate** che verrà utilizzato di default nell'addestramento dei modelli, a meno di modifiche particolari basate sull'analisi dei grafici delle **accuracy** e **loss** del **training set** e **validation set**.

L'**early stopping** è una tecnica utilizzata per fermare l'addestramento quando il modello smette di migliorare sul set di validazione, evitando il rischio di **overfitting**. In particolare, l'addestramento viene interrotto quando la metrica di validazione (ad esempio, l'accuracy o la loss) non migliora per un numero prestabilito di epoche.

Il **decadimento del learning rate** è una strategia in cui il learning rate viene ridotto progressivamente durante l'addestramento. Questo aiuta a stabilizzare la convergenza del modello, soprattutto nelle fasi finali, per evitare che il modello salti soluzioni ottimali a causa di un learning rate troppo alto. Il decadimento può essere implementato in vari modi, come per esempio tramite **reduce on plateau** (cioè quando dopo n epoche la val_loss non migliora) o una funzione che diminuisce il learning rate in modo esponenziale o stepwise.

```

early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True,
)

decay_learning_rate = tf.keras.callbacks.ReduceLROnPlateau(
    monitor='val_loss', factor=0.1, patience=5, min_lr=1e-6
)

```

✓ 4.5. Modello sulle forme d'onda grezze

Come già descritto all'inizio del capitolo, per quanto riguarda il modello che riceve in input direttamente la **forma grezza** dell'audio, si è scelto di optare inizialmente nell'implementazione della **WaveNet** vista a lezione.

Essendo questa una rete neurale progettata per la **generazione di audio**, è stata leggermente modificata per adattarsi al compito di **classificazione**. Le modifiche principali riguardano l'architettura finale del modello, che è stata adattata per produrre output di classificazione anziché generare audio, pur mantenendo l'intuizione e la struttura della WaveNet.

```
from tensorflow.keras.models import Model

# Rimane invariato rispetto al blocco residuale della waveNet per la generazione
def residual_block(x, filters, kernel_size, dilation_rate):
    # Convoluzione causale dilatata
    res = layers.Conv1D(filters, kernel_size, dilation_rate=dilation_rate, padding='causal')(x)
    res = layers.ReLU()(res)

    # Convoluzione successiva
    res = layers.Conv1D(filters, kernel_size, dilation_rate=dilation_rate, padding='causal')(res)
    res = layers.ReLU()(res)

    # Connessione residua
    res = layers.Add()([x, res]) # Sommatoria dell'input con l'output
    res = layers.ReLU()(res) # Attivazione finale

    return res

def build_wavenet(num_blocks, filters, kernel_size, dilation_rates):
    inputs = layers.Input((MAX_DURATION*TARGET_SR,))
    reshape_input = layers.Reshape((MAX_DURATION * TARGET_SR, 1))(inputs)

    # Primo strato di convoluzione per mappare l'input
    x = layers.Conv1D(filters, kernel_size, padding="causal")(reshape_input)

    # Creazione di una serie di blocchi residuali
    for i in range(num_blocks):
        dilation_rate = dilation_rates[i % len(dilation_rates)] # Cicla tra dilation_rates
        x = residual_block(x, filters, kernel_size, dilation_rate)

    # Strato finale per la classificazione
    x = layers.GlobalAveragePooling1D()(x) # Pooling globale per ridurre la dimensione
    x = layers.Dense(64, activation="relu")(x) # Un layer fully connected
    output = layers.Dense(3, activation="softmax")(x) # Softmax per classificazione

    # Generazione del modello
    model = Model(inputs, output)
```

```
return model
```

```
# Configurazioni
```

```
input_shape = (None, 1) # Input con una sequenza di campioni (audio a campione s
```

```
num_classes = 10 # Numero di classi per la classificazione
```

```
num_blocks = 10 # Numero di blocchi residuali
```

```
filters = 64 # Numero di filtri per convoluzione
```

```
kernel_size = 2 # Dimensione del kernel
```

```
dilation_rates = [1, 2, 4, 8, 16] # Dilation rates
```

```
wavenet = build_wavenet(num_blocks, filters, kernel_size, dilation_rates)
```

```
compile_and_fit(wavenet, train_ds, validation_ds, epochs=300, optimizer="adam", c
```

 Model: "functional_1"

Layer (type)	Output Shape	Param #	Conn
input_layer_5 (InputLayer)	(None, 48000)	0	-
reshape_4 (Reshape)	(None, 48000, 1)	0	input_layer_5
conv1d_44 (Conv1D)	(None, 48000, 64)	192	reshape_4
conv1d_45 (Conv1D)	(None, 48000, 64)	8,256	conv1d_44
re_lu_40 (ReLU)	(None, 48000, 64)	0	conv1d_45
conv1d_46 (Conv1D)	(None, 48000, 64)	8,256	re_lu_40
add_20 (Add)	(None, 48000, 64)	0	conv1d_46 conv1d_45
re_lu_41 (ReLU)	(None, 48000, 64)	0	add_20
conv1d_47 (Conv1D)	(None, 48000, 64)	8,256	re_lu_41
re_lu_42 (ReLU)	(None, 48000, 64)	0	conv1d_47
conv1d_48 (Conv1D)	(None, 48000, 64)	8,256	re_lu_42
add_21 (Add)	(None, 48000, 64)	0	re_lu_42 conv1d_48
re_lu_43 (ReLU)	(None, 48000, 64)	0	add_21
conv1d_49 (Conv1D)	(None, 48000, 64)	8,256	re_lu_43
re_lu_44 (ReLU)	(None, 48000, 64)	0	conv1d_49
conv1d_50 (Conv1D)	(None, 48000, 64)	8,256	re_lu_44
add_22 (Add)	(None, 48000, 64)	0	re_lu_44 conv1d_50
re_lu_45 (ReLU)	(None, 48000, 64)	0	add_22
conv1d_51 (Conv1D)	(None, 48000, 64)	8,256	re_lu_45
re_lu_46 (ReLU)	(None, 48000, 64)	0	conv1d_51
conv1d_52 (Conv1D)	(None, 48000, 64)	8,256	re_lu_46
add_23 (Add)	(None, 48000, 64)	0	re_lu_46 conv1d_52
re_lu_47 (ReLU)	(None, 48000, 64)	0	add_23
conv1d_53 (Conv1D)	(None, 48000, 64)	8,256	re_lu_47
re_lu_48 (ReLU)	(None, 48000, 64)	0	conv1d_53
conv1d_54 (Conv1D)	(None, 48000, 64)	8,256	re_lu_48
add_24 (Add)	(None, 48000, 64)	0	re_lu_48 conv1d_54
re_lu_49 (ReLU)	(None, 48000, 64)	0	add_24

conv1d_55 (Conv1D)	(None, 48000, 64)	8,256	re_1
re_lu_50 (ReLU)	(None, 48000, 64)	0	conv
conv1d_56 (Conv1D)	(None, 48000, 64)	8,256	re_1
add_25 (Add)	(None, 48000, 64)	0	re_1 conv
re_lu_51 (ReLU)	(None, 48000, 64)	0	add_
conv1d_57 (Conv1D)	(None, 48000, 64)	8,256	re_1
re_lu_52 (ReLU)	(None, 48000, 64)	0	conv
conv1d_58 (Conv1D)	(None, 48000, 64)	8,256	re_1
add_26 (Add)	(None, 48000, 64)	0	re_1 conv
re_lu_53 (ReLU)	(None, 48000, 64)	0	add_
conv1d_59 (Conv1D)	(None, 48000, 64)	8,256	re_1
re_lu_54 (ReLU)	(None, 48000, 64)	0	conv
conv1d_60 (Conv1D)	(None, 48000, 64)	8,256	re_1
add_27 (Add)	(None, 48000, 64)	0	re_1 conv
re_lu_55 (ReLU)	(None, 48000, 64)	0	add_
conv1d_61 (Conv1D)	(None, 48000, 64)	8,256	re_1
re_lu_56 (ReLU)	(None, 48000, 64)	0	conv
conv1d_62 (Conv1D)	(None, 48000, 64)	8,256	re_1
add_28 (Add)	(None, 48000, 64)	0	re_1 conv
re_lu_57 (ReLU)	(None, 48000, 64)	0	add_
conv1d_63 (Conv1D)	(None, 48000, 64)	8,256	re_1
re_lu_58 (ReLU)	(None, 48000, 64)	0	conv
conv1d_64 (Conv1D)	(None, 48000, 64)	8,256	re_1
add_29 (Add)	(None, 48000, 64)	0	re_1 conv
re_lu_59 (ReLU)	(None, 48000, 64)	0	add_
global_average_pooling1d... (GlobalAveragePooling1D)	(None, 64)	0	re_1
dense_4 (Dense)	(None, 64)	4,160	glob
dense_5 (Dense)	(None, 3)	195	dens

Total params: 169,667 (662.76 KB)

Trainable params: 169,667 (662.76 KB)

Trainable params: 105,000 (105.00 MB)

Non-trainable params: 0 (0.00 B)

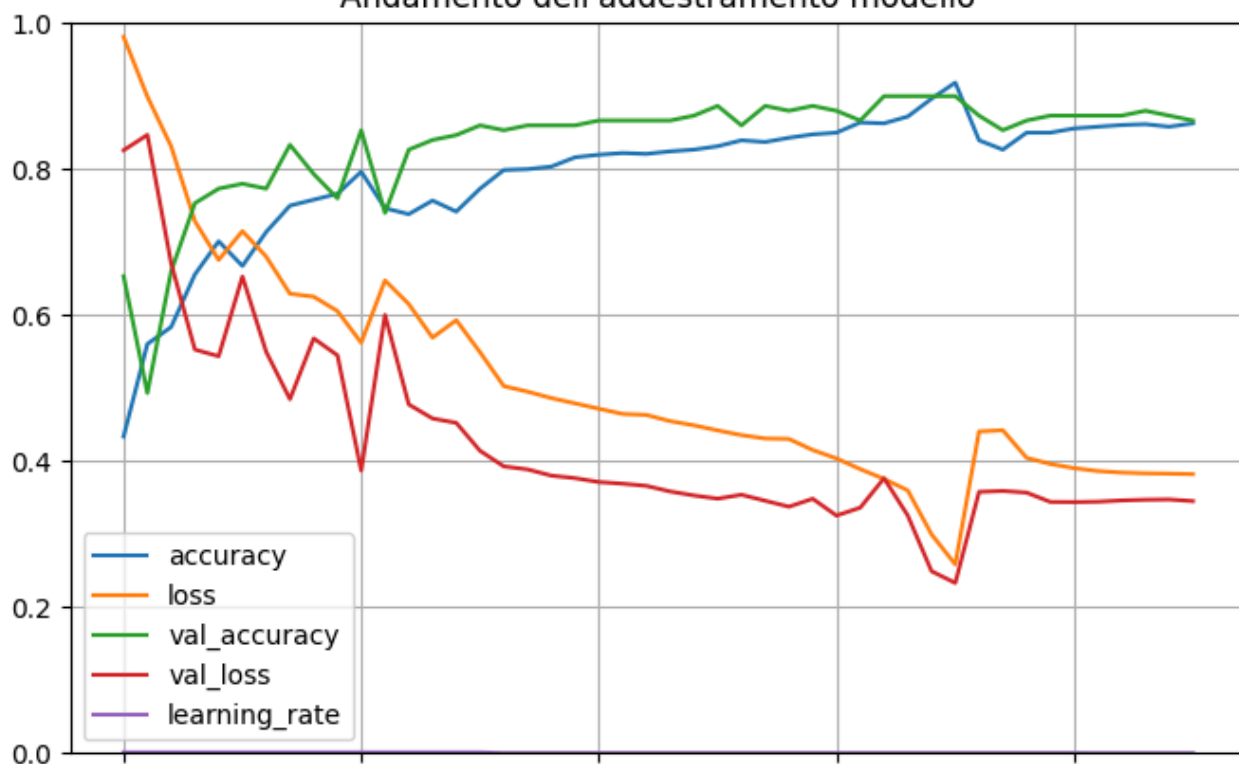
Epoch 1/300					
27/27	—————	140s	5s/step	- accuracy: 0.3575	- loss: 1.0281 -
Epoch 2/300					
27/27	—————	132s	5s/step	- accuracy: 0.5612	- loss: 0.9015 -
Epoch 3/300					
27/27	—————	123s	5s/step	- accuracy: 0.5702	- loss: 0.8548 -
Epoch 4/300					
27/27	—————	123s	5s/step	- accuracy: 0.6120	- loss: 0.7817 -
Epoch 5/300					
27/27	—————	123s	5s/step	- accuracy: 0.6947	- loss: 0.6838 -
Epoch 6/300					
27/27	—————	123s	5s/step	- accuracy: 0.6846	- loss: 0.6696 -
Epoch 7/300					
27/27	—————	142s	5s/step	- accuracy: 0.6767	- loss: 0.7060 -
Epoch 8/300					
27/27	—————	143s	5s/step	- accuracy: 0.7320	- loss: 0.6194 -
Epoch 9/300					
27/27	—————	123s	5s/step	- accuracy: 0.7573	- loss: 0.6283 -
Epoch 10/300					
27/27	—————	143s	5s/step	- accuracy: 0.7624	- loss: 0.5945 -
Epoch 11/300					
27/27	—————	141s	5s/step	- accuracy: 0.7787	- loss: 0.5836 -
Epoch 12/300					
27/27	—————	124s	5s/step	- accuracy: 0.7493	- loss: 0.6206 -
Epoch 13/300					
27/27	—————	142s	5s/step	- accuracy: 0.7079	- loss: 0.6677 -
Epoch 14/300					
27/27	—————	123s	5s/step	- accuracy: 0.7471	- loss: 0.5953 -
Epoch 15/300					
27/27	—————	124s	5s/step	- accuracy: 0.7434	- loss: 0.5783 -
Epoch 16/300					
27/27	—————	142s	5s/step	- accuracy: 0.7539	- loss: 0.5676 -
Epoch 17/300					
27/27	—————	142s	5s/step	- accuracy: 0.7734	- loss: 0.5388 -
Epoch 18/300					
27/27	—————	142s	5s/step	- accuracy: 0.8158	- loss: 0.4636 -
Epoch 19/300					
27/27	—————	123s	5s/step	- accuracy: 0.7953	- loss: 0.4944 -
Epoch 20/300					
27/27	—————	142s	5s/step	- accuracy: 0.8174	- loss: 0.4686 -
Epoch 21/300					
27/27	—————	123s	5s/step	- accuracy: 0.8099	- loss: 0.4980 -
Epoch 22/300					
27/27	—————	123s	5s/step	- accuracy: 0.8005	- loss: 0.4852 -
Epoch 23/300					
27/27	—————	123s	5s/step	- accuracy: 0.8083	- loss: 0.4635 -
Epoch 24/300					
27/27	—————	123s	5s/step	- accuracy: 0.8115	- loss: 0.4791 -
Epoch 25/300					
27/27	—————	142s	5s/step	- accuracy: 0.8336	- loss: 0.4423 -
Epoch 26/300					
27/27	—————	123s	5s/step	- accuracy: 0.8317	- loss: 0.4477 -
Epoch 27/300					
27/27	—————	124s	5s/step	- accuracy: 0.8525	- loss: 0.4140 -
Epoch 28/300					
27/27	—————	142s	5s/step	- accuracy: 0.8288	- loss: 0.4462 -
Epoch 29/300					
27/27	—————	123s	5s/step	- accuracy: 0.8472	- loss: 0.4508 -
Epoch 30/300					

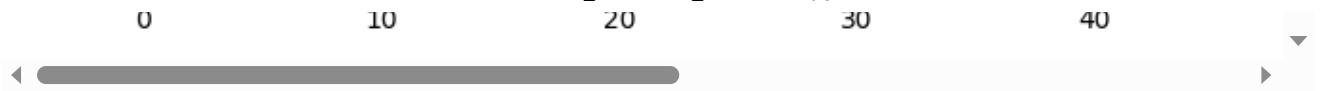
```

27/27 ----- 124s 5s/step - accuracy: 0.8556 - loss: 0.4048 -
Epoch 31/300
27/27 ----- 141s 5s/step - accuracy: 0.8379 - loss: 0.4198 -
Epoch 32/300
27/27 ----- 123s 5s/step - accuracy: 0.8430 - loss: 0.4302 -
Epoch 33/300
27/27 ----- 123s 5s/step - accuracy: 0.8692 - loss: 0.3503 -
Epoch 34/300
27/27 ----- 142s 5s/step - accuracy: 0.8248 - loss: 0.4370 -
Epoch 35/300
27/27 ----- 142s 5s/step - accuracy: 0.8947 - loss: 0.3020 -
Epoch 36/300
27/27 ----- 123s 5s/step - accuracy: 0.9147 - loss: 0.2645 -
Epoch 37/300
27/27 ----- 123s 5s/step - accuracy: 0.8796 - loss: 0.3415 -
Epoch 38/300
27/27 ----- 142s 5s/step - accuracy: 0.8125 - loss: 0.4598 -
Epoch 39/300
27/27 ----- 142s 5s/step - accuracy: 0.8418 - loss: 0.4379 -
Epoch 40/300
27/27 ----- 123s 5s/step - accuracy: 0.8520 - loss: 0.3773 -
Epoch 41/300
27/27 ----- 124s 5s/step - accuracy: 0.8444 - loss: 0.3981 -
Epoch 42/300
27/27 ----- 141s 5s/step - accuracy: 0.8533 - loss: 0.3770 -
Epoch 43/300
27/27 ----- 123s 5s/step - accuracy: 0.8301 - loss: 0.4070 -
Epoch 44/300
27/27 ----- 123s 5s/step - accuracy: 0.8701 - loss: 0.3783 -
Epoch 45/300
27/27 ----- 123s 5s/step - accuracy: 0.8426 - loss: 0.3991 -
Epoch 46/300
27/27 ----- 123s 5s/step - accuracy: 0.8633 - loss: 0.3651 -
Training loss: 0.25843527913093567
Training accuracy: 0.9186046719551086
Validation loss: 0.2330137938261032
Validation accuracy: 0.8999999761581421

```

Andamento dell'addestramento modello





I risultati, come ci si poteva aspettare non sono dei migliori visto che la waveNet è una rete con molti parametri e profonda e questo la rende difficile da addestrare con un dataset con pochi meno di 1000 campioni come il nostro.

✓ 4.5.2 Modello `slim_waveNet`

Dopo aver implementato il modello WaveNet modificato per la classificazione audio, ho cercato di sviluppare una versione ottimizzata, denominata da me `slim_waveNet`, che mantenga l'intuizione di base della WaveNet, ovvero l'utilizzo di **strati convoluzionali con dilatazione crescente e padding causale**, ma con l'obiettivo di rendere il **training più veloce e ridurre il numero di parametri** utilizzati.

Il modello `slim_waveNet` si propone di **semplificare l'architettura** della WaveNet, mantenendo però l'intuizione di base delle convoluzioni causali con dilatazione crescente. In particolare, le modifiche introdotte includono:

1. **Eliminazione delle connessioni residue:** Una delle principali differenze rispetto alla WaveNet classica è l'eliminazione di connessioni residue tra i vari strati, riducendo la complessità computazionale e migliorando la velocità di addestramento.
2. **Riduzione dei parametri:** In `slim_waveNet`, sono stati ridotti il numero di filtri e le dimensioni dei kernel nei vari strati convoluzionali. Mantenendo la dilatazione crescente, ma con una struttura più snella, il modello riduce significativamente il numero di parametri senza compromettere la capacità di apprendimento delle caratteristiche temporali nei segnali audio.
3. **Ottimizzazione del training:** Grazie alla riduzione delle connessioni e dei parametri, `slim_waveNet` è progettato per addestrarsi più velocemente rispetto alla WaveNet classica, mantenendo prestazioni competitive nella generazione e classificazione di segnali audio.

In sintesi, mentre la **WaveNet classica** è molto potente per la generazione di audio grazie alla sua architettura profonda e alla dilatazione crescente, il `slim_waveNet` cerca di ottimizzare il processo di addestramento, riducendo i parametri e accelerando il training, senza sacrificare la capacità di catturare le dinamiche temporali cruciali per la generazione di audio. Si cerca quindi di semplificare un modello nato per la generazione di audio ad una rete ottimizzata per svolgere un compito sicuramente più semplice che è quello della classificazione.

```
# Modello che prende spunto dalla waveNet ma ha l'obiettivo di dimezzare il numero di parametri
slim_waveNet = models.Sequential([
    layers.Input(shape=(MAX_DURATION*TARGET_SR,)),
    layers.Reshape((MAX_DURATION*TARGET_SR, 1)),
    # Primo layer di convoluzione causale con dilatazione 1
    layers.Conv1D(64, kernel_size=2, dilation_rate=1, padding='causal', activation='relu')
])
```

```
# Strati convoluzionali con dilatazione crescente intervallati da livelli di
layers.Conv1D(64, kernel_size=2, dilation_rate=2, padding='causal', activation='relu')
layers.Conv1D(64, kernel_size=2, dilation_rate=4, padding='causal', activation='relu')
layers.MaxPooling1D(pool_size=16),
layers.Dropout(0.2),

layers.Conv1D(64, kernel_size=2, dilation_rate=8, padding='causal', activation='relu')
layers.Conv1D(64, kernel_size=2, dilation_rate=16, padding='causal', activation='relu')
layers.MaxPooling1D(pool_size=16),
layers.Dropout(0.2),

# Layer finali di convoluzioni senza dilation rate (per ottenere embedding)
layers.Conv1D(128, kernel_size=1, activation='relu'),
layers.Conv1D(128, kernel_size=1, activation='relu'),
layers.MaxPooling1D(pool_size=16),
layers.Dropout(0.2),

# Strato di pooling per ridurre la dimensionalità
layers.GlobalMaxPooling1D(),

layers.Dense(64, activation='relu'),
layers.Dropout(0.2),

# Strato fully connected per la classificazione finale
layers.Dense(3, activation='softmax')
])
```

```
compile_and_fit(slim_waveNet, train_ds, validation_ds, epochs=300, optimizer="ada
```

 **Model: "sequential"**

Layer (type)	Output Shape	
reshape_5 (Reshape)	(None, 48000, 1)	
conv1d_65 (Conv1D)	(None, 48000, 64)	
conv1d_66 (Conv1D)	(None, 48000, 64)	
conv1d_67 (Conv1D)	(None, 48000, 64)	
max_pooling1d (MaxPooling1D)	(None, 3000, 64)	
dropout (Dropout)	(None, 3000, 64)	
conv1d_68 (Conv1D)	(None, 3000, 64)	
conv1d_69 (Conv1D)	(None, 3000, 64)	
max_pooling1d_1 (MaxPooling1D)	(None, 187, 64)	
dropout_1 (Dropout)	(None, 187, 64)	
conv1d_70 (Conv1D)	(None, 187, 128)	
conv1d_71 (Conv1D)	(None, 187, 128)	
max_pooling1d_2 (MaxPooling1D)	(None, 11, 128)	
dropout_2 (Dropout)	(None, 11, 128)	
global_max_pooling1d (GlobalMaxPooling1D)	(None, 128)	
dense_6 (Dense)	(None, 64)	
dropout_3 (Dropout)	(None, 64)	
dense_7 (Dense)	(None, 3)	

Total params: 66,499 (259.76 KB)
Trainable params: 66,499 (259.76 KB)
Non-trainable params: 0 (0.00 B)

Epoch 1/300

27/27 ————— 29s 670ms/step - accuracy: 0.3815 - loss: 1.0499

Epoch 2/300

27/27 ————— 25s 342ms/step - accuracy: 0.6147 - loss: 0.8906

Epoch 3/300

27/27 ————— 10s 345ms/step - accuracy: 0.6132 - loss: 0.8113

Epoch 4/300

27/27 ————— 10s 346ms/step - accuracy: 0.6713 - loss: 0.7270

Epoch 5/300

27/27 ————— 10s 342ms/step - accuracy: 0.7256 - loss: 0.6197

Epoch 6/300

27/27 ————— 10s 340ms/step - accuracy: 0.7197 - loss: 0.6482

Epoch 7/300

27/27 ————— 10s 340ms/step - accuracy: 0.7502 - loss: 0.5898

Epoch 8/300

27/27 ————— 10s 340ms/step - accuracy: 0.7658 - loss: 0.5643

Epoch 9/300

27/27 ————— 10s 341ms/step - accuracy: 0.7641 - loss: 0.5640

Epoch 10/300

```
Epoch 10/300  
27/27 ----- 10s 343ms/step - accuracy: 0.7980 - loss: 0.4990  
Epoch 11/300  
27/27 ----- 10s 343ms/step - accuracy: 0.8233 - loss: 0.4491  
Epoch 12/300  
27/27 ----- 20s 341ms/step - accuracy: 0.7728 - loss: 0.5335  
Epoch 13/300  
27/27 ----- 10s 342ms/step - accuracy: 0.8325 - loss: 0.4304  
Epoch 14/300  
27/27 ----- 10s 345ms/step - accuracy: 0.8603 - loss: 0.3748  
Epoch 15/300  
27/27 ----- 21s 340ms/step - accuracy: 0.8299 - loss: 0.4446  
Epoch 16/300  
27/27 ----- 10s 343ms/step - accuracy: 0.8446 - loss: 0.3867  
Epoch 17/300  
27/27 ----- 10s 345ms/step - accuracy: 0.8625 - loss: 0.3467  
Epoch 18/300  
27/27 ----- 10s 343ms/step - accuracy: 0.9032 - loss: 0.2433  
Epoch 19/300  
27/27 ----- 10s 346ms/step - accuracy: 0.8992 - loss: 0.2467  
Epoch 20/300  
27/27 ----- 10s 341ms/step - accuracy: 0.8806 - loss: 0.2797  
Epoch 21/300  
27/27 ----- 20s 340ms/step - accuracy: 0.9053 - loss: 0.2461  
Epoch 22/300  
27/27 ----- 10s 342ms/step - accuracy: 0.9095 - loss: 0.2202  
Epoch 23/300  
27/27 ----- 10s 344ms/step - accuracy: 0.9235 - loss: 0.1891  
Epoch 24/300  
27/27 ----- 10s 344ms/step - accuracy: 0.9170 - loss: 0.2113  
Epoch 25/300  
27/27 ----- 11s 344ms/step - accuracy: 0.9072 - loss: 0.1992  
Epoch 26/300  
27/27 ----- 20s 339ms/step - accuracy: 0.9229 - loss: 0.1774  
Epoch 27/300  
27/27 ----- 10s 341ms/step - accuracy: 0.9036 - loss: 0.2470  
Epoch 28/300  
27/27 ----- 10s 347ms/step - accuracy: 0.8958 - loss: 0.2425  
Epoch 29/300  
27/27 ----- 10s 344ms/step - accuracy: 0.9318 - loss: 0.1767  
Epoch 30/300  
27/27 ----- 10s 345ms/step - accuracy: 0.9229 - loss: 0.1814  
Epoch 31/300  
27/27 ----- 10s 342ms/step - accuracy: 0.9413 - loss: 0.1457  
Epoch 32/300  
27/27 ----- 10s 341ms/step - accuracy: 0.9533 - loss: 0.1282  
Epoch 33/300  
27/27 ----- 10s 341ms/step - accuracy: 0.9471 - loss: 0.1392  
Epoch 34/300  
27/27 ----- 10s 341ms/step - accuracy: 0.9502 - loss: 0.1268  
Epoch 35/300  
27/27 ----- 10s 341ms/step - accuracy: 0.9431 - loss: 0.1112  
Epoch 36/300  
27/27 ----- 20s 341ms/step - accuracy: 0.9332 - loss: 0.1425  
Epoch 37/300  
27/27 ----- 20s 343ms/step - accuracy: 0.9529 - loss: 0.1199  
Epoch 38/300  
27/27 ----- 10s 346ms/step - accuracy: 0.9552 - loss: 0.1251  
Epoch 39/300  
27/27 ----- 20s 341ms/step - accuracy: 0.9631 - loss: 0.1042  
Epoch 40/300
```

27/27	_____	10s	342ms/step	-	accuracy: 0.9523	-	loss: 0.1316
Epoch 41/300							
27/27	_____	10s	343ms/step	-	accuracy: 0.9581	-	loss: 0.1050
Epoch 42/300							
27/27	_____	10s	346ms/step	-	accuracy: 0.9579	-	loss: 0.1205
Epoch 43/300							
27/27	_____	20s	341ms/step	-	accuracy: 0.9442	-	loss: 0.1198
Epoch 44/300							
27/27	_____	20s	343ms/step	-	accuracy: 0.9482	-	loss: 0.1408
Epoch 45/300							
27/27	_____	21s	348ms/step	-	accuracy: 0.9524	-	loss: 0.1280
Epoch 46/300							
27/27	_____	10s	346ms/step	-	accuracy: 0.9581	-	loss: 0.1113
Epoch 47/300							
27/27	_____	10s	345ms/step	-	accuracy: 0.9526	-	loss: 0.1198
Epoch 48/300							
27/27	_____	10s	341ms/step	-	accuracy: 0.9501	-	loss: 0.1227
Epoch 49/300							
27/27	_____	10s	341ms/step	-	accuracy: 0.9591	-	loss: 0.1099
Epoch 50/300							
27/27	_____	10s	340ms/step	-	accuracy: 0.9582	-	loss: 0.1120
Epoch 51/300							
27/27	_____	10s	339ms/step	-	accuracy: 0.9576	-	loss: 0.1182
Epoch 52/300							
27/27	_____	10s	342ms/step	-	accuracy: 0.9535	-	loss: 0.1175
Epoch 53/300							
27/27	_____	10s	343ms/step	-	accuracy: 0.9461	-	loss: 0.1125
Epoch 54/300							
27/27	_____	10s	344ms/step	-	accuracy: 0.9620	-	loss: 0.1006
Epoch 55/300							
27/27	_____	10s	348ms/step	-	accuracy: 0.9597	-	loss: 0.1008
Epoch 56/300							
27/27	_____	21s	340ms/step	-	accuracy: 0.9506	-	loss: 0.1194
Epoch 57/300							
27/27	_____	20s	341ms/step	-	accuracy: 0.9680	-	loss: 0.1100
Epoch 58/300							
27/27	_____	10s	344ms/step	-	accuracy: 0.9507	-	loss: 0.1160
Epoch 59/300							
27/27	_____	10s	345ms/step	-	accuracy: 0.9735	-	loss: 0.0873
Epoch 60/300							
27/27	_____	10s	344ms/step	-	accuracy: 0.9628	-	loss: 0.0986
Epoch 61/300							
27/27	_____	20s	340ms/step	-	accuracy: 0.9606	-	loss: 0.0975
Epoch 62/300							
27/27	_____	10s	341ms/step	-	accuracy: 0.9691	-	loss: 0.1001
Epoch 63/300							
27/27	_____	10s	347ms/step	-	accuracy: 0.9661	-	loss: 0.0941
Epoch 64/300							
27/27	_____	21s	343ms/step	-	accuracy: 0.9676	-	loss: 0.0862
Epoch 65/300							
27/27	_____	10s	344ms/step	-	accuracy: 0.9553	-	loss: 0.1138
Epoch 66/300							
27/27	_____	10s	346ms/step	-	accuracy: 0.9586	-	loss: 0.0937
Epoch 67/300							
27/27	_____	21s	345ms/step	-	accuracy: 0.9598	-	loss: 0.1041
Epoch 68/300							
27/27	_____	21s	341ms/step	-	accuracy: 0.9556	-	loss: 0.1010
Epoch 69/300							
27/27	_____	10s	343ms/step	-	accuracy: 0.9638	-	loss: 0.1070
Epoch 70/300							

```

21/21 _____ 10s 346ms/step - accuracy: 0.9639 - loss: 0.1047
Epoch 71/300
27/27 _____ 10s 349ms/step - accuracy: 0.9610 - loss: 0.0942
Epoch 72/300
27/27 _____ 21s 344ms/step - accuracy: 0.9567 - loss: 0.0901
Epoch 73/300
27/27 _____ 20s 341ms/step - accuracy: 0.9665 - loss: 0.0928
Epoch 74/300
27/27 _____ 10s 342ms/step - accuracy: 0.9688 - loss: 0.0841
Epoch 75/300
27/27 _____ 10s 345ms/step - accuracy: 0.9528 - loss: 0.1042
Epoch 76/300
27/27 _____ 21s 342ms/step - accuracy: 0.9755 - loss: 0.0721
Epoch 77/300
27/27 _____ 10s 344ms/step - accuracy: 0.9687 - loss: 0.0850
Epoch 78/300
27/27 _____ 10s 344ms/step - accuracy: 0.9679 - loss: 0.0792
Epoch 79/300
27/27 _____ 10s 343ms/step - accuracy: 0.9734 - loss: 0.0841
Epoch 80/300
27/27 _____ 10s 347ms/step - accuracy: 0.9657 - loss: 0.0947
Epoch 81/300
27/27 _____ 20s 340ms/step - accuracy: 0.9723 - loss: 0.0882
Epoch 82/300
27/27 _____ 10s 341ms/step - accuracy: 0.9567 - loss: 0.1048
Epoch 83/300
27/27 _____ 10s 343ms/step - accuracy: 0.9691 - loss: 0.0820
Epoch 84/300
27/27 _____ 10s 346ms/step - accuracy: 0.9639 - loss: 0.0775
Epoch 85/300
27/27 _____ 10s 344ms/step - accuracy: 0.9638 - loss: 0.0826
Epoch 86/300
27/27 _____ 10s 342ms/step - accuracy: 0.9647 - loss: 0.0842
Epoch 87/300
27/27 _____ 20s 339ms/step - accuracy: 0.9681 - loss: 0.0809
Epoch 88/300
27/27 _____ 10s 341ms/step - accuracy: 0.9721 - loss: 0.0825
Epoch 89/300
27/27 _____ 10s 343ms/step - accuracy: 0.9679 - loss: 0.0866
Epoch 90/300
27/27 _____ 10s 345ms/step - accuracy: 0.9663 - loss: 0.0779

```

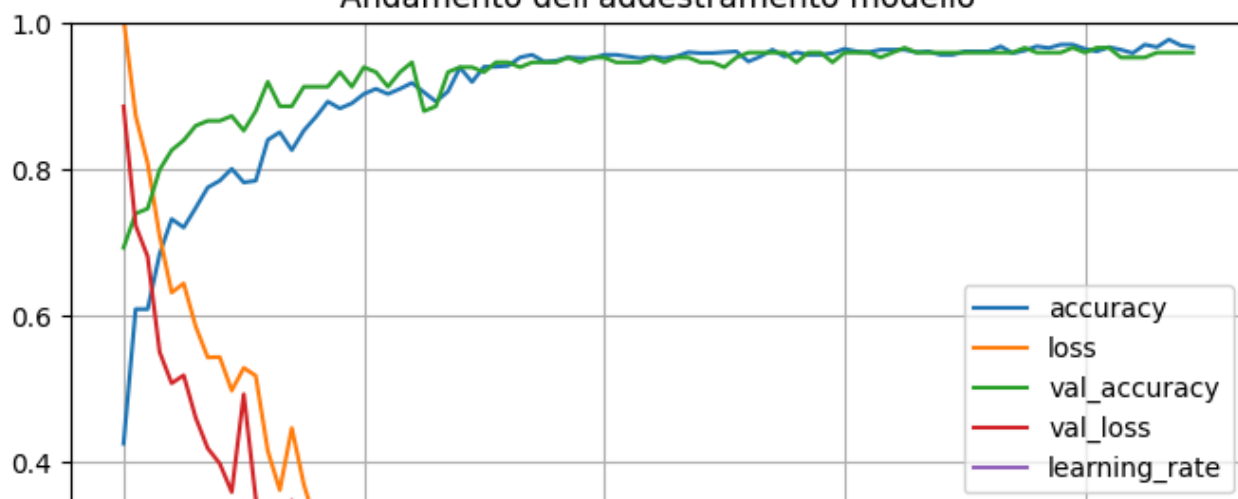
Training loss: 0.08348724246025085

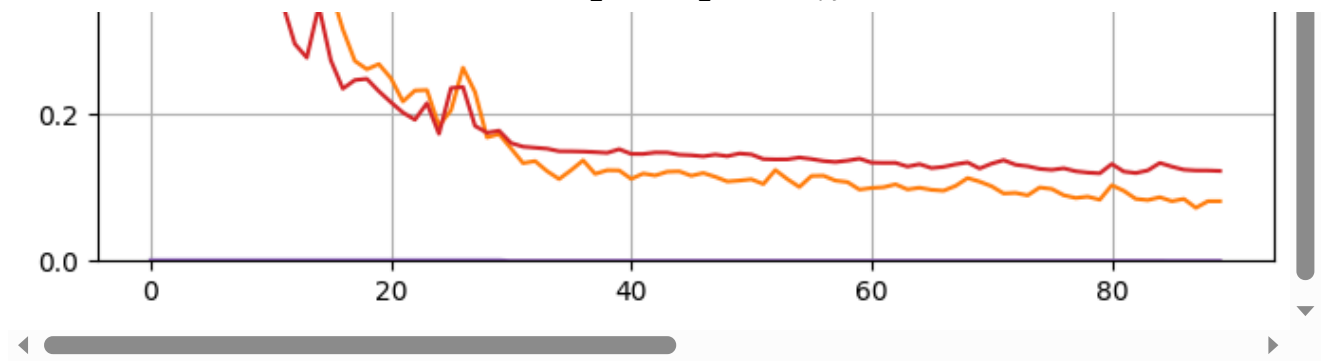
Training accuracy: 0.9709302186965942

Validation loss: 0.11949945986270905

Validation accuracy: 0.9666666388511658

Andamento dell'addestramento modello





✓ 4.6. Modello sugli spettrogrammi

✓ 4.6.1. Stabilire l'input

Il primo passo per creare un modello che riceve in input gli spettrogrammi è analizzare le diverse rappresentazioni degli spettrogrammi di un audio. Gli spettrogrammi sono rappresentazioni visive delle frequenze presenti in un segnale audio nel tempo, ottenute attraverso la trasformata di Fourier. Questi permettono di ottenere una visione più chiara delle caratteristiche del segnale audio, che possono essere utilizzate per il successivo processo di addestramento del modello.

Per questa analisi, sono state definite due funzioni differenti per stampare e visualizzare due tipologie di spettrogrammi:

1. **Spettrogramma STFT (Short-Time Fourier Transform):** La funzione

`plot_spectrogram_STFT` stampa lo spettrogramma in **scala lineare STFT**. La STFT è una trasformata che divide il segnale audio in segmenti temporali, analizzando la frequenza contenuta in ciascun segmento.

2. **Spettrogramma di Mel:** La seconda funzione `plot_spectrogram_MEL` stampa lo **spettrogramma di Mel**, che è una rappresentazione del segnale audio che utilizza la **scala Mel** per la frequenza. La scala Mel è una scala di frequenza non lineare progettata per imitare la percezione umana del suono, in cui le frequenze più basse sono rappresentate in modo più dettagliato rispetto a quelle più alte. Questo tipo di spettrogramma è molto utilizzato per compiti legati al riconoscimento vocale e alla sintesi del parlato, poiché è più vicino alla percezione uditiva umana.

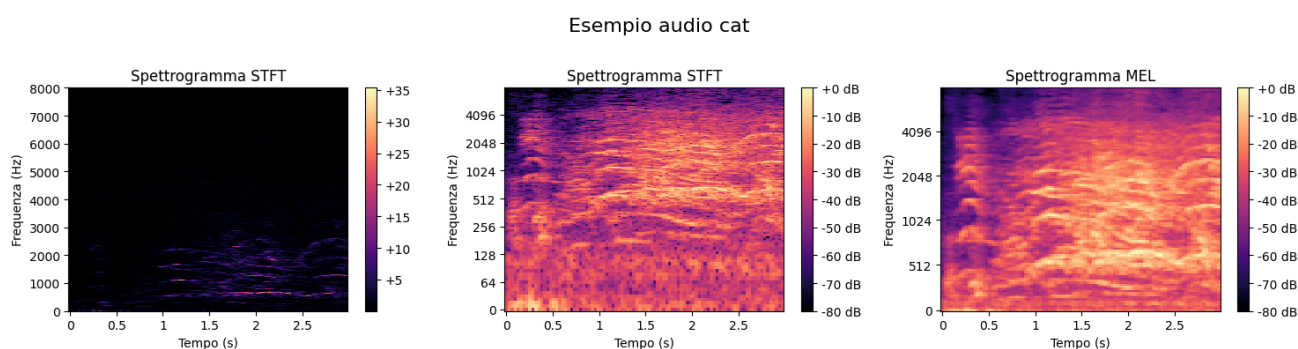
```
# Funzione per disegnare lo spettrogramma
def plot_spectrogram_STFT(audio, sr, fig, ax, title="Spettrogramma STFT", y_scale
D = librosa.stft(audio)
D_db = librosa.amplitude_to_db(np.abs(D), ref=np.max)
img = librosa.display.specshow(D_db if y_scale == "log" else np.abs(D), sr=sr
fig.colorbar(img, ax=ax, format='%+2.0f dB' if y_scale == "log" else '%+2.0f'
ax.set_title(title)
ax.set_xlabel("Tempo (s)")
ax.set_ylabel("Frequenza (Hz)")

def plot_spectrogram_MEL(audio, sr, fig, ax, title="Spettrogramma MEL"):
mel_spectrogram = librosa.feature.melspectrogram(y=audio, sr=sr)
mel_spectrogram_db = librosa.power_to_db(mel_spectrogram, ref=np.max)
img = librosa.display.specshow(mel_spectrogram_db, sr=sr, x_axis="time", y_ax
fig.colorbar(img, ax=ax, format='%+2.0f dB')
ax.set_title(title)
ax.set_xlabel("Tempo (s)")
ax.set_ylabel("Frequenza (Hz)")
```


Per eseguire queste due funzioni andiamo a campionare randomicamente un audio dal nostro training set

```
for x_batch, y_batch in train_ds.take(1): # Prende il primo batch
    random_index = np.random.randint(len(x_batch)) # indice randomico
    audio_example = x_batch[random_index].numpy().squeeze() # Prendi l'audio
    label_example = class_names[np.argmax(y_batch[random_index].numpy())] # Pren
```

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
fig.suptitle(f"Esempio audio {label_example}", fontsize=16)
plot_spectrogram_STFT(audio_example, TARGET_SR, fig, axes[0], y_scale="linear")
plot_spectrogram_STFT(audio_example, TARGET_SR, fig, axes[1], y_scale="log")
plot_spectrogram_MEL(audio_example, TARGET_SR, fig, axes[2])
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```



```
D = librosa.stft(audio_example)
print(D.shape)
mel_spectrogram = librosa.feature.melspectrogram(y=audio_example, sr=TARGET_SR)
print(mel_spectrogram.shape)
```



```
(1025, 94)
(128, 94)
```

Come possiamo vedere, lo **spettrogramma STFT** in **scala lineare** perde molte caratteristiche. Considerando che gran parte dello spettrogramma risulta essere nero, si ha una dimensione grande dell'immagine con **dati "inutili"**.

In **scala logaritmica**, invece, otteniamo uno spettrogramma molto più ricco di informazioni, poiché le differenze di intensità vengono enfatizzate. L'analisi dimensionale, inoltre, ci permette di osservare come l'utilizzo dello **spettrogramma MEL** ci consenta di mantenere un alto grado di informazione, ma a una dimensione notevolmente ridotta, migliorando l'efficienza computazionale senza perdere dettagli importanti.

Inoltre, un altro lato positivo è che esiste un **layer in Keras** apposito per la generazione di uno spettrogramma a partire da una forma d'onda grezza. In questo modo, possiamo utilizzare gli stessi set di dati impiegati nei modelli precedenti anche per i modelli basati su spettrogrammi.

Il vantaggio aggiuntivo è che, durante l'inferenza, è possibile fornire direttamente l'**audio grezzo** al modello per effettuare la predizione, senza la necessità di pre-processare ulteriormente l'audio in uno spettrogramma, rendendo il processo più semplice e veloce.

```
LayerMelSpectrogram = tf.keras.layers.MelSpectrogram(
    fft_length=2048,          # Lunghezza della FFT, permette una buona ris
    sequence_stride=512,      # Passo tra i frame (stride). 512 offre un b
    sequence_length=None,     # Lunghezza della sequenza in input (lasciato
    window="hann",            # Finestra di tipo Hann, spesso usata per rid
    sampling_rate=16000,      # Frequenza di campionamento del segnale audi
    num_mel_bins=128,         # Numero di bande Mel. 128 è una scelta comun
    min_freq=20.0,            # Frequenza minima. Per i versi di animali, 2
    max_freq=None,            # Frequenza massima. Nessun limite specifico,
    power_to_db=True,         # Applicare la conversione in decibel (dB), u
    top_db=80.0,              # Ridurre il range dinamico, imposta il massi
    mag_exp=2.0,              # Elevato il quadrato della magnitudine per
    min_power=1e-10,          # Valore minimo di potenza per evitare log(0)
    ref_power=1.0             # Normalizzazione della potenza rispetto a qu
)
```

✓ 4.6.2. Modello **simil_AlexNet**

Dopo aver testato la **AlexNet** vista a lezione con risultati pessimi anche in questo caso, supponendo che il compito di classificazione rispetto al dataset **MNIST** fosse molto più semplice, ho provveduto a modificare la rete tramite tentativi logici con l'obiettivo di **semplificare la rete e diminuire il numero di parametri**, considerando la dimensione ridotta del dataset di training rispetto a MNIST.

La rete prodotta dopo svariati tentativi è stata chiamata **simil_AlexNet** e porta con sé le seguenti principali modifiche:

Struttura dei Layer Convoluzionali

- **AlexNet** : 5 strati convoluzionali con filtri crescenti (96, 256, 384, 384, 256) e kernel più grandi (11x11, 5x5, 3x3).
- **simil** : 3 strati convoluzionali con filtri più piccoli (32, 64, 128) e kernel più piccoli (3x3, 5x5).

Strati Fully Connected (Dense)

- **AlexNet** : 2 strati densi da 4096 neuroni ciascuno.
- **simil** : 2 strati densi da 64 e 32 neuroni, con dropout.

Numero di Parametri

- **AlexNet** : Maggiore numero di parametri a causa della profondità e dimensione dei layer convoluzionali e fully connected.
- **simil** : Numero inferiore di parametri, più efficiente dal punto di vista computazionale.

Complesso vs Leggero

- **AlexNet** : Modello più complesso, adatto a task complessi, maggiore capacità di apprendimento ma maggiore rischio di overfitting.
- **simil** : Modello più leggero, più veloce da addestrare, adatto a task più semplici.

Tempo di Addestramento e Risorse

- **AlexNet** : Richiede più risorse computazionali e tempo di addestramento.
- **simil** : Più rapido nell'addestramento e adatto a hardware meno potente.

```
# Modello di partenza AlexNet rivisitato per eliminare molti parametri
simil_AlexNet = models.Sequential([
    layers.Input(shape=(MAX_DURATION*TARGET_SR,)),
    LayerMelSpectrogram,
    layers.Reshape((128, 94, 1)),

    # Primo blocco convoluzionale
    layers.Conv2D(32, kernel_size=(3, 3), strides=(2, 2), activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(pool_size=(3, 3), strides=(2, 2)),

    # Secondo blocco convoluzionale
    layers.Conv2D(64, kernel_size=(5, 5), padding="same", activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(pool_size=(3, 3), strides=(2, 2)),

    # Terzo blocco convoluzionale
    layers.Conv2D(128, kernel_size=(3, 3), padding="same", activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D(pool_size=(3, 3), strides=(2, 2)),

    # Flatten per passare ai livelli completamente connessi (Dense)
    layers.Flatten(),

    # Primo Fully Connected Layer
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.5), # Per ridurre overfitting

    # Secondo Fully Connected Layer
    layers.Dense(32, activation='relu'),
    layers.Dropout(0.5),
```

```
# Ultimo strato (Softmax per classificazione su N classi)
layers.Dense(3, activation='softmax')
])
```

```
compile_and_fit(simil_AlexNet, train_ds, validation_ds, epochs=300, optimizer="ad
```

 Model: "sequential"

Layer (type)	Output Shape	
mel_spectrogram (MelSpectrogram)	(None, 128, 94)	
reshape (Reshape)	(None, 128, 94, 1)	
conv2d (Conv2D)	(None, 63, 46, 32)	
batch_normalization (BatchNormalization)	(None, 63, 46, 32)	
max_pooling2d (MaxPooling2D)	(None, 31, 22, 32)	
conv2d_1 (Conv2D)	(None, 31, 22, 64)	
batch_normalization_1 (BatchNormalization)	(None, 31, 22, 64)	
max_pooling2d_1 (MaxPooling2D)	(None, 15, 10, 64)	
conv2d_2 (Conv2D)	(None, 15, 10, 128)	
batch_normalization_2 (BatchNormalization)	(None, 15, 10, 128)	
max_pooling2d_2 (MaxPooling2D)	(None, 7, 4, 128)	
flatten (Flatten)	(None, 3584)	
dense (Dense)	(None, 64)	
dropout (Dropout)	(None, 64)	
dense_1 (Dense)	(None, 32)	
dropout_1 (Dropout)	(None, 32)	
dense_2 (Dense)	(None, 3)	

Total params: 357,955 (1.37 MB)

Trainable params: 357,507 (1.36 MB)

Non-trainable params: 448 (1.75 KB)

Epoch 1/300

27/27  84s 283ms/step - accuracy: 0.5021 - loss: 2.1220

Epoch 2/300

27/27  1s 11ms/step - accuracy: 0.6838 - loss: 0.8305 -

Epoch 3/300

27/27  1s 12ms/step - accuracy: 0.7431 - loss: 0.6331 -

Epoch 4/300

27/27  2s 13ms/step - accuracy: 0.8112 - loss: 0.4613 -

Epoch 5/300

27/27  3s 14ms/step - accuracy: 0.8321 - loss: 0.4682 -

Epoch 6/300

27/27  2s 14ms/step - accuracy: 0.8604 - loss: 0.3904 -

Epoch 7/300

27/27  2s 14ms/step - accuracy: 0.8654 - loss: 0.3430 -

Epoch 8/300

27/27  3s 15ms/step - accuracy: 0.8956 - loss: 0.2914 -

Epoch 9/300

27/27  1s 12ms/step - accuracy: 0.8870 - loss: 0.3381 -

Epoch 10/300

```

Epoch 10/300
27/27 ----- 1s 17ms/step - accuracy: 0.8981 - loss: 0.2955 -
Epoch 11/300
27/27 ----- 3s 14ms/step - accuracy: 0.9127 - loss: 0.1841 -
Epoch 12/300
27/27 ----- 2s 14ms/step - accuracy: 0.9253 - loss: 0.2284 -
Epoch 13/300
27/27 ----- 2s 14ms/step - accuracy: 0.9400 - loss: 0.1718 -
Epoch 14/300
27/27 ----- 2s 12ms/step - accuracy: 0.9553 - loss: 0.1210 -
Epoch 15/300
27/27 ----- 2s 11ms/step - accuracy: 0.9564 - loss: 0.1378 -
Epoch 16/300
27/27 ----- 1s 10ms/step - accuracy: 0.9490 - loss: 0.1730 -
Epoch 17/300
27/27 ----- 1s 10ms/step - accuracy: 0.9309 - loss: 0.1847 -
Epoch 18/300
27/27 ----- 2s 17ms/step - accuracy: 0.9377 - loss: 0.1614 -
Epoch 19/300
27/27 ----- 2s 17ms/step - accuracy: 0.9539 - loss: 0.1069 -
Epoch 20/300
27/27 ----- 3s 13ms/step - accuracy: 0.9716 - loss: 0.0761 -
Epoch 21/300
27/27 ----- 2s 15ms/step - accuracy: 0.9664 - loss: 0.1052 -
Epoch 22/300
27/27 ----- 2s 15ms/step - accuracy: 0.9749 - loss: 0.0722 -
Epoch 23/300
27/27 ----- 1s 13ms/step - accuracy: 0.9718 - loss: 0.1365 -
Epoch 24/300
27/27 ----- 2s 12ms/step - accuracy: 0.9462 - loss: 0.1324 -
Epoch 25/300
27/27 ----- 2s 20ms/step - accuracy: 0.9841 - loss: 0.0586 -
Epoch 26/300
27/27 ----- 2s 17ms/step - accuracy: 0.9864 - loss: 0.0612 -
Epoch 27/300
27/27 ----- 2s 14ms/step - accuracy: 0.9897 - loss: 0.0346 -
Epoch 28/300
27/27 ----- 2s 11ms/step - accuracy: 0.9797 - loss: 0.0504 -
Epoch 29/300
27/27 ----- 1s 11ms/step - accuracy: 0.9789 - loss: 0.0530 -
Epoch 30/300
27/27 ----- 1s 12ms/step - accuracy: 0.9843 - loss: 0.0413 -
Epoch 31/300
27/27 ----- 3s 14ms/step - accuracy: 0.9879 - loss: 0.0409 -
Epoch 32/300
27/27 ----- 1s 12ms/step - accuracy: 0.9841 - loss: 0.0461 -
Epoch 33/300
27/27 ----- 2s 13ms/step - accuracy: 0.9809 - loss: 0.0521 -
Epoch 34/300
27/27 ----- 3s 13ms/step - accuracy: 0.9876 - loss: 0.0317 -
Epoch 35/300
27/27 ----- 2s 12ms/step - accuracy: 0.9870 - loss: 0.0343 -
Epoch 36/300
27/27 ----- 1s 12ms/step - accuracy: 0.9758 - loss: 0.0446 -
Epoch 37/300
27/27 ----- 1s 10ms/step - accuracy: 0.9822 - loss: 0.0426 -
Epoch 38/300
27/27 ----- 1s 10ms/step - accuracy: 0.9831 - loss: 0.0392 -
Epoch 39/300
27/27 ----- 1s 11ms/step - accuracy: 0.9859 - loss: 0.0434 -
Epoch 40/300

```

27/27 ————— **1s** 10ms/step - accuracy: 0.9860 - loss: 0.0303 -
Epoch 41/300
27/27 ————— **1s** 13ms/step - accuracy: 0.9859 - loss: 0.0375 -
Epoch 42/300
27/27 ————— **3s** 15ms/step - accuracy: 0.9846 - loss: 0.0519 -
Epoch 43/300
27/27 ————— **2s** 14ms/step - accuracy: 0.9963 - loss: 0.0227 -
Epoch 44/300
27/27 ————— **1s** 12ms/step - accuracy: 0.9832 - loss: 0.0453 -
Epoch 45/300
27/27 ————— **2s** 15ms/step - accuracy: 0.9883 - loss: 0.0321 -
Epoch 46/300
27/27 ————— **2s** 18ms/step - accuracy: 0.9763 - loss: 0.0505 -
Training loss: 0.04796989634633064
Training accuracy: 0.9767441749572754
Validation loss: 0.011206287890672684
Validation accuracy: 0.9933333396911621

